Where the Capital Bound Moves:

Settlement Objects, Proof Economics, and Agent Coordination on Bitcoin Payment Networks

Patrick B. Dugan*

October 2026
Draft for technical review, revised October 21, 2026

Abstract

Bitcoin payment systems commit capital under different rules for spending and recovery. We express those rules as prefix conservation constraints and compare the capital-time required to serve a fixed population of software agents. A Lightning channel partitions liquidity by edge and direction. Ark pools some provisioning decisions at a server while adding the capital that the server fronts until spent outputs expire. Proposed multiparty channels pool balances within a signing group, and the LayerTree covenant proposal exchanges a shorter provisioning horizon for on-chain bytes and contention on a shared output. We distinguish these accounting constraints from achievable delivery: routing decisions change later balances, and a cut bound alone supplies no numerical ceiling on the value of routing information. Archived simulator campaigns on sampled public topology find a 0.35 percentage-point gain from the tested online heuristic over retries, a 1.46-point gain from the tested oracle over retries, and a 6.13-point gain from demand-aware placement under stationary hotspots. These estimates are conditional on the simulator and workload. A separate reference model compares settlement objects under explicit forecasts and recovery clocks; expiry-time recovery follows current Second documentation, while next-round recovery is a counterfactual sensitivity. We give a claim-once reward construction for canonical single-signer credentials, identify its epoch and service-measurement requirements, and examine the observations available when agents share control. Protocol-specific execution, real proof costs, and budget-constrained comparisons at a held-out service target remain experiments. The accompanying code includes finite counterexamples and a reproducible audit of the archived numerical results.

1 Introduction

Software agents that transact over Bitcoin need funds in a form their counterparties can accept. A successful payment can consume directional channel balance, require a server to advance funds, or wait for an on-chain transaction. Those costs depend on where capital sits and when it becomes reusable. This paper asks how the settlement object changes that accounting for a population that makes frequent payments from small balances.

On Lightning, the balance of each channel belongs to a particular edge and side. We express this fact as a prefix cut identity: every sequence of settled payments must preserve nonnegative directional balances. Routing chooses which feasible sequence occurs and

*Spiral program. Model, fixtures, and experiment protocols: https://github.com/patrickdugan/Spiral. Correspondence: patrick.buchanan.dugan@gmail.com.

therefore affects future liquidity. The identity constrains all policies, while a comparison between policies requires a workload and an execution model. Section 3 keeps these questions separate and reports the archived routing and placement comparisons with their uncertainty.

The comparison between settlement objects then asks where provisioning moves. Section 4 models the server's pooled stock and its fronted, unrecovered value. Current Second documentation specifies expiry-time recovery after successful Lightning sends; the much shorter next-round case is retained as a counterfactual. Section 5 analyzes a proposed multiparty channel, where a hub can pool capacity within each group whose members sign its updates. LayerTree is a further proposed object with per-member credit leaves and recurring chain costs. Its queueing examples require an explicit service assumption. These proposed objects are analyzed under their stated semantics and remain subject to implementation and consensus changes.

Sections 6 and 7 develop a prospective reward and monitoring layer. A canonical credential and an epoch snapshot can prevent repeated claims on the same capital. A service ratio also requires a complete, authenticated denominator. Neither a payment-success estimate nor authenticated reward weight determines a coalition's total profit. The observer model makes those remaining questions explicit.

The evidence has separate origins. Table 1 reanalyzes archived campaigns on a July 2023 public gossip snapshot with synthetic capacities and balances. The settlement comparisons come from a small TypeScript reference model. The registry uses a simulated verifier. Historical testnet4 headers support a header-integrity audit and an audit of miner timestamp behavior, while payment execution and recovery latency have yet to be measured. The repository records the baseline, the October 21 corrections, and commands for reproducing each result [16]. Section 8 defines the experiments that would connect these models to implementations.

2 Related work

Lightning liquidity and routing. Pickhardt and Richter [38] formulate payment delivery over hidden balances as a minimum-cost flow problem under a uniform prior on channel balance and show large gains in reliability from splitting; Pickhardt and Nowostawski treat imbalance and rebalancing [37]. Both take the channel graph and its capacities as given and optimize what crosses it. Guasoni, Huberman, and Shikhelman [22] model the channel itself as a capital decision under bilateral demand, which is the decision this paper says is binding for an agent population. The Lightning Service Provider specifications [29] standardize just-in-time channel opening, which moves the inbound decision to the moment of first receipt but keeps it per channel. The prefix cut bound of Section 3 is a conservation statement of the kind that underlies max-flow min-cut [20], applied to cumulative settled crossings rather than to a single flow.

Privacy and probing. That channel balances are only nominally hidden is established by Herrera-Joancomartí et al. [23], Tikhomirov et al. [52], and Nisslmueller et al. [35]; that the network's privacy against a well-placed observer is partial is established empirically by Kappos et al. [25] and Romiti et al. [39], and Béres et al. [5] gives a traffic analysis of the economics. We use these as the observer rows for the channel object in Section 7; the contribution there is to add the rows for objects that have a server.

Ark and covenants. Ark was proposed by Keceli [26] as a server-coordinated off-chain output scheme; its current parameters are those published by its implementers [45, 46, 49] and summarized by Bitcoin Optech [7]. The implementer's own simulation [45] compares an Ark server's liquidity requirement to an LSP's by top-up frequency and finds the comparison flips with it; Section 4 names the quantity behind that flip and adds a term. Covenant proposals would change the channel object itself. The earliest are the template commitment of Rubin [41] and the eltoo channel of Decker et al. [14], surveyed by Bitcoin Optech [8]. The current one is BIP 448 [43], a draft that bundles a Taproot-native template hash [42], signature verification on stack messages, and a push of the internal key, and whose stated purpose is rebindable signatures: LN-Symmetry channels, in which any later state replaces any earlier one and no penalty is needed [9]; the multiparty channels and factories [10] that such symmetry makes practical; and Ark variants whose rounds need no interaction [40]. Section 5 works out what that does to the bound. LayerTree [36], a draft of October 2026, is a covenant shared output of a different design: a Merkle-sum tree committed to one output, every change enforced by tapscript, no pre-signed interior, no expiry of the output, a unilateral exit of one transaction and $O(D)$ bytes, in four opcode tiers none of which is active, and vouchers in place of Ark's out-of-round transfers. Sections 4 and 5.6 place it. The results of Sections 3 and 4 depend on nothing in it.

Sybil bounds and nullifiers. The Sybil problem is Douceur [15]'s: without a trusted authority, identities are free. Babaioff et al. [2] give Sybil-proof reward schemes for a propagation setting; the registry of Section 6 is instead a proof-of-exposure scheme, in which a claimant proves control of a settled object without revealing which. The nullifier it uses is Zerocash's [4, 24], transplanted from spend-once to claim-once, and the gap we find in the transplant is that spending is the one thing a nullifier is designed to allow.

Optimistic verification on Bitcoin. Bitcoin consensus verifies no succinct proofs. BitVM [30, 31] and BitVM3 [32] make a statement enforceable by an assert-and-disprove game under a committee-at-setup and one-honest-challenger assumption; we use the published BitVM3 cost figures as inputs to the escrow object and to the economics of Section 6. Groth16 [21] is the proof system those figures assume.

Covert channels, wardens, and collusion. The warden is Simmons [50]'s, and the capacity-versus-detectability framing of a steganographic channel is Cachin [11]'s; Section 7 argues the capacity question is second-order for this setting. Wash-trade detection [13, 53] supplies the netting statistics whose failure modes the reference model reproduces. On what a population of agents can do without a message, Calvano et al. [12] show algorithmic pricers converging on supra-competitive prices without communication; Stigler [51] located cartel instability in the unenforceability of side-agreements, which is the lever an enforceable escrow pulls; and the focal-point account of coordination without communication is Schelling [44]'s.

3 Lightning as a graph: the directional capital bound

3.1 Setting

Let $G = (N, E)$ be a directed multigraph. Each channel $i \in E$ joins two nodes u_i, v_i and has capacity χ_i, split at any time into balances $\ell_i(u_i), \ell_i(v_i) \geq 0$ with $\ell_i(u_i) + \ell_i(v_i) = \chi_i$. A

node can spend over channel i only from its own side. The balances are hidden from all but the two endpoints; the capacities are public through gossip [28]. A payment of amount a from s to t is an attempt along a path; it settles only if every edge on the path has $\ell_i(\text{upstream}) \geq a$ at the moment of the attempt, in which case a moves from the upstream to the downstream side of every edge on the path, and otherwise it fails and nothing moves. The reference ledger ignores routing fees; with fees, each cut crossing must use the amount actually transferred on that edge. Funding, splicing and closing change capacity through on-chain events with confirmation clocks. A liquidity lease is a contract whose underlying channel or balance event determines its settlement clock; off-chain rebalancing changes directional balances without necessarily changing capacity.

For a node set $A \subseteq N$, write ∂A for the channels with exactly one endpoint in A. After n settled events, let $O_A(n)$ and $I_A(n)$ be the cumulative amounts that have crossed ∂A outward and inward, counting each crossing of a path that leaves and re-enters; let $S_A(n) = \sum_{i \in \partial A} \ell_i(A\text{-side endpoint})$ be A's outbound stock on the cut and $S'_A(n) = \sum_{i \in \partial A} \chi_i - S_A(n)$ its inbound stock; and let $K_A(n)$ be the net stock added to S_A by capacity events, with $K'_A(n)$ the same for S'_A.

3.2 The cut bound

Lemma 1 (prefix cut bound)**.** *For every $A \subseteq N$ and every n,*

$$O_A(n) - I_A(n) \;\leq\; S_A(0) + K_A(n), \qquad I_A(n) - O_A(n) \;\leq\; S'_A(0) + K'_A(n).$$

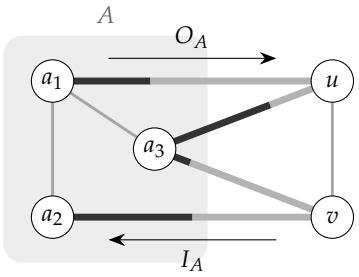
Proof. A settled crossing of channel $i \in \partial A$ outward moves its amount from the A-side balance to the other side, decreasing S_A by that amount and increasing S'_A by the same; an inward crossing does the reverse; a capacity event changes S_A by its contribution to K_A. Events inside A or outside A touch no channel in ∂A. So $S_A(n) = S_A(0) - O_A(n) + I_A(n) + K_A(n)$, and $S_A(n) \geq 0$ because balances are nonnegative. The second inequality is the same identity on S'_A. □

The inequalities are prefix conditions: they must hold at every n, not only at the horizon, and they bind on whichever stock is being consumed. A population of agents that mostly pays outward binds on the first with I_A small, so on K_A, which is to say on someone having funded outbound from A. A population that mostly gets paid binds on the second, on K'_A, which is inbound toward A and must be put there by A's counterparties, from their side, by a forecast. Figure 1(a) draws the cut.

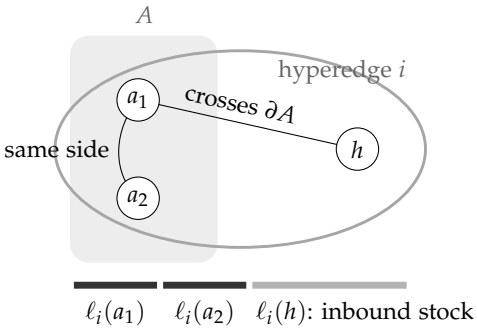
Proposition 1 (policy-independent accounting)**.** *Lemma 1 holds under every routing policy, including one with exact balance knowledge. For fixed initial stocks and capacity events, each policy's settled history satisfies the same form of prefix constraint. The constraint alone does not order the volume delivered by different policies.*

Proof. The identity follows from balance conservation on each settled crossing and non-negativity after each event. Neither step requires a rule for selecting attempts. Different policies can settle different paths and leave different balances for the next request. Their future feasible paths can therefore differ even when their attempt budgets agree and their earlier attempts all succeed. □

The distinction matters for interpreting the experiments. A feasible path can consume liquidity that a later request needs, while another feasible path preserves it. The finite witness in `audit/peer_review/` exhibits this effect with two payments. Thus an oracle



(a) channels on ∂A. Dark: the A-side balance; light: the far side. S_A sums the dark parts, S'_A the light.



(b) the same cut through a hyperedge. χ_i is divided among the members; a transfer within one side moves nothing across ∂A.

Figure 1: The prefix cut bound. (a) On a graph, each channel crossing ∂A is one capacity split between its two endpoints. The A-side balances sum to the outbound stock S_A and the far-side balances to the inbound stock S'_A; a settled crossing moves value from one sum to the other, so $S_A(n) = S_A(0) - O_A(n) + I_A(n) + K_A(n) \geq 0$ under any routing policy (Lemma 1). (b) On a hypergraph a channel's capacity is divided among all its members. A crossing changes the two sums as before, and a transfer between two members on the same side changes neither, so the bound is unchanged (Lemma 2); for the agents inside A the inbound stock on channel i is the hub's part alone, one number for the group.

comparison is specific to the tested policies, costs and demand. A cut bound supplies an accounting necessity; a routing claim also needs a feasible execution or an optimization argument.

3.3 The provisioning problem has no pooling

The quantity that Lemma 1 makes the operator choose is K'_A: how much inbound to put toward each agent a before a's demand is known. Let D_a be a's demand process over a horizon H, and let $M_a = \max_{t \leq H} B_a(t)$ be its peak holdings in a reference replay with unlimited inbound capacity. Here B_a is evaluated after each event: a receipt credits its full value, and an unaffordable outgoing request is rejected in full. Thus M_a includes the receipt-before-spend peak within a round. A receipt that would push a's balance above the provisioned inbound fails. The operator chooses $\hat{f}_a$ before seeing D_a and pays for idle capital if $\hat{f}_a > M_a$ and for failed receipts if $\hat{f}_a < M_a$: a newsvendor problem [1], one per channel.

What makes the problem expensive for a population is not that each instance is hard but that there are N of them and they do not pool. Capacity provisioned toward agent a sits on channel a and serves no other agent, so the total inbound an operator must carry is $\sum_a \hat{f}_a$, a sum of forecast peaks, regardless of how the agents' demands are correlated. If the same N agents drew on a common pool, the operator would carry a forecast of the peak of the sum, $\hat{f}_S$, and for independent imbalances the two differ by a factor that the following remark sizes.

Remark 1 (scale of the pooling gain). The scaling depends on the process being pooled. For independent zero-drift signed imbalances in a diffusion approximation, an individual peak is of order $\sigma\sqrt{H}$ and the peak of their sum is of order $\sigma\sqrt{NH}$. The ratio of the sum of expected peaks to the expected aggregate peak is then of order $\sqrt{N}$. This comparison

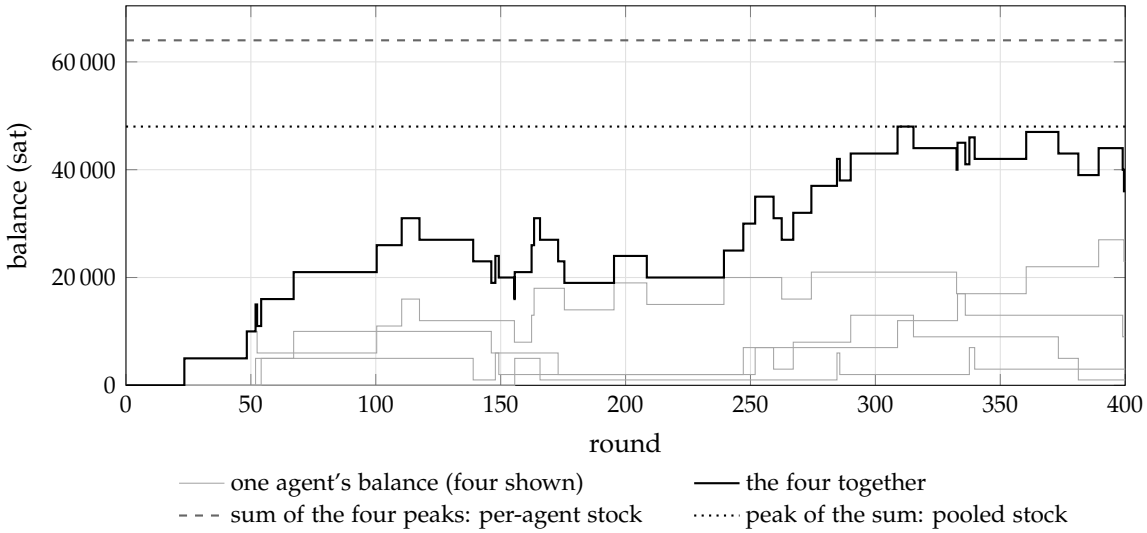


Figure 2: Four agents with zero offered drift, drawn with the model's generator over 400 rounds. Each receipt precedes that agent's spend; an unaffordable spend is rejected in full. Fractional positions within a round expose this event order. The paths show the resulting finite jump processes, with every state-changing event prefix retained. The dashed line is the sum of individual event-prefix peaks; the dotted line is the peak of aggregate holdings. These are the separate and pooled oracle stocks for this trace with zero safety margin. Their ratio measures this trace's pooling gain; the diffusion comparison in Remark 1 uses additional assumptions. Generated by `model/grid.ts`.

assumes enough initial balances to permit the negative excursions. Common accumulating drift gives both quantities order $N\mu H$ and can dominate the fluctuation benefit.

For a precise zero-start benchmark, let $R_a(t)$ be independent reflected Brownian motions with volatility σ and no drift. Their marginal process law is that of $|\sigma B_a(t)|$. Write

$$Q_N = \frac{\sum_a \mathbb{E}[\sup_{t \le H} R_a(t)]}{\mathbb{E}[\sup_{t \le H} \sum_a R_a(t)]}.$$

The numerator is $N\sigma\sqrt{\pi H/2}$. The denominator is at least $N\sigma\sqrt{2H/\pi}$, by evaluating at H, and at most the numerator, so $1 \le Q_N \le \pi/2$. The law of large numbers for the continuous paths, with integrable supremum, gives $Q_N \to \pi/2$ as $N \to \infty$ because $\mathbb{E}R_a(t)$ increases with t.

This is a statement about a ratio of expectations in a reflected-diffusion model. A finite jump process that rejects an unaffordable spend in full has different boundary dynamics; an individual simulated sample need not obey the same bound or vary monotonically with group size. The reference model therefore treats $\pi/2$ as a benchmark and tests its own pooling ratios directly. A draining drift and correlations also change the comparison. Figure 2 illustrates finite paths from the model.

3.4 What a simulator shows

The original, replication and precision campaigns [16] contain 4,536 archived rows, of which 3,348 belong to the sealed evaluation arms. Their combined analysis uses 28 sampled-topology clusters from one July 16, 2023 gossip snapshot, with synthetic capacities and hidden balances. Paired contrasts are averaged within each cluster and then

Intervention and comparator	Gain (pp)	95% interval (pp)
Bounded retries over one-shot public routing	5.14	[3.08, 7.20]
Online terminal-feedback heuristic over retries	0.35	[0.15, 0.55]
Balance oracle over the online heuristic	1.11	[0.64, 1.58]
Balance oracle over retries (recomputed contrast)	1.46	[0.84, 2.07]
Failure-aware placement over random, stationary demand	6.13	[4.91, 7.35]
Failure-aware placement over random, shifted demand	0.44	[−0.21, 1.09]

Table 1: Archived public-topology simulator contrasts. Percentage points refer to settled-payment success. The routing policies use a catalog of up to eight simple paths. Public routing attempts at most one path, retry and online policies attempt up to three, and the oracle selects a feasible path, if one exists, after inspecting hidden feasibility across the catalog. The October 21 audit reproduces the stored contrasts and computes oracle over retries directly from paired rows. These are comparisons between particular policies, rather than upper bounds on routing or placement.

equally across clusters. The intervals in Table 1 use the archived two-sided Student-t procedure on those cluster means. They describe uncertainty conditional on this snapshot and campaign design; sampled clusters can overlap and do not constitute independent networks.

The original, replication and precision campaigns contribute 4, 6 and 18 evaluation clusters, respectively. The precision extension was designed after heterogeneity appeared in the replication and specified tests on combined data. These combined intervals are nominal summaries of an adaptive extension, without adjustment for that adaptation or for testing several contrasts. Campaign-specific estimates are retained in the audit.

The tested online heuristic adds a small positive effect beyond retries, with its interval inside the ± 1 percentage-point practical-equivalence band declared in the precision-extension configuration. The shifted-demand placement interval crosses zero and extends beyond that band, so it supports neither an exact zero effect nor practical equivalence under that rule. The stationary placement effect is larger than the tested routing effects on these workloads. Payment-success points measure delivery, rather than revenue, profit, or the optimum over all policies.

The earlier inference audit also found dependence on Python hash order in graph traversal. The archived rows and their analysis can be verified byte-for-byte; exact campaign regeneration requires the missing process-level ordering to be pinned. Its later free-failure diagnostic compared different attempt schedules and cannot decompose the archived information effect into an attempt-budget effect. A new implementation campaign must record the node ordering, random seed and permitted observations alongside the attempt budget.

3.5 The ghost-node apparatus

The campaigns used a graph-rewrite construction from earlier work in a clearing setting [17]: augment the graph with a nonphysical node or edge, solve a balancing problem in the augmented graph, and map the solution back to permissible operations, here to funding, splicing, or leasing on a real edge. In the channel setting the construction is a solver for K'_A: a ghost edge is a unit of inbound the solver wishes existed, and compiling it is the act

of forecasting. Section 4 will move that solver one tier up.

4 Ark: where the bound moves

4.1 Settlement objects as typed resources

The channel is one way to hold value off-chain. To compare it with others on the terms the lemma cares about, we need each object described by what it costs to exit, who can hurt its holder, what the holder must do to keep it safe, and when its status changes.

Definition 1 (settlement object)**.** A settlement object is a tuple $\rho = (\mathcal{A}, v, F, X, \Lambda, \Gamma, \tau)$: a spend authority $\mathcal{A}$ (a script or a signer set), a value v, a finality clock F mapping elapsed time or block height to a settled or pending status, a unilateral exit X with a cost and a delay, a liveness obligation Λ the holder carries to keep the object trustless, a set Γ of parties whose misbehavior can cost the holder value or access, and an expiry τ after which the object's trust assumptions change.

Table 2 instantiates the tuple for the on-chain output U, the directional channel balance C, the Ark virtual output V, and the optimistic escrow E, with a multiparty channel C_k and a native-covenant escrow E' for comparison, neither available today. The Ark parameters are those published by Second as of September 2026 [46, 49]: a virtual output (VTXO) is an off-chain output in a tree whose root is an on-chain round transaction; it is spendable by the holder and server together, or by the holder alone after a relative timelock, by broadcasting its branch; it expires after a server-configurable lifetime, 28 days by default, before which the holder must refresh it in a round; and an out-of-round transfer is a pending state the recipient trusts until the next round confirms [7].

Three things the table makes visible. Of the objects available today, the channel is the only one whose value is direction-partitioned: a VTXO of value v is spendable to any counterparty the server will co-sign for, so there is no inbound side to provision at the holder's tier. The VTXO and the LayerTree leaf are the objects with an obligation on the holder's calendar, with a difference: the VTXO's holder refreshes or forfeits to the server, the leaf's renews or is paid out on chain. Either obligation is one an always-online process meets at no marginal cost, as it also meets the channel's breach-watching; liveness distinguishes agents from humans, not one object from another. And the escrow is the only object whose finality is a game rather than a count, with a committee rather than a counterparty in Γ.

The LayerTree leaf. The fourth object is a draft and not a deployment [36]. Its state is a leaf, an amount with an owner key, an exit key, an evictor key, a lease and a counter, at a position in an off-chain Merkle-sum tree of depth D; the output commits to the root through its internal key, the predicates are its taptree, and every spend proves the changed positions against the committed root and recreates the output under the new one. Authorisation is the draft's Table 4: the owner exits at any time, co-signs updates, issues vouchers and carves its leaf; the evictor pays a leaf out in full after its lease; the operator grafts, co-signs, evicts, detaches a cohort and fills empty positions, and cannot move or reduce another leaf, delay an exit beyond the draft's assumption [C] (Section 4.7), or freeze value it did not graft and fund; anyone redeems a valid voucher. The exit is one transaction and $O(D)$ bytes with no timelock: 1,020 vB at $D = 16$ in the FULL tier on the draft's transaction model (1,030 serialised), 762 to 1,149 vB across $D = 8$ to 20, 1,355 in MID, 1,473 in

	$\mathcal{A}$	F	X	Λ	Γ	τ
U on-chain output	script	k confirmations	none	none	consensus only	∞
C channel balance	2-of-2 + HTLC	HTLC settle, bilateral	commitment tx + `to_self_delay`	watch for breach	counterparty liveness	∞, direction-locked
V Ark VTXO	holder $\wedge$ server (MuSig2), or holder after CSV	round confirmation; out-of-round pending	branch + leaf broadcast, CSV 144 ($\approx$24 h)	refresh before τ	server (censor); sender + server (collude on pending)	28 d, server-set
E optimistic escrow	release after Δ, or slash on disprove	challenge period Δ	on-chain already	≥ 1 honest challenger online	setup committee (1-of-n deletes key); proof-system setup	program-defined
C_k multiparty channel	n-of-n of members (MuSig2), rebindable	update signed by all members	latest state, then settle after CSV	every member signs each update; watch for stale state	every co-member (availability)	∞
T LayerTree leaf	owner key; Merkle-sum path to the root in the output key	the spend confirms; no timelock	one tx, $O(D)$ bytes, 1,020 vB for FULL at $D = 16$; owner online, or a tower	renew the lease or be paid out; a fee coin for most exits	operator (graft, co-sign, evict after lease, paying in full; fill the empty positions a partial voucher needs); a voucher's issuer until confirmed	none on the output; lease per leaf, eviction pays the member
E' covenant escrow	template-enforced script	template match	on-chain already	none	consensus only	program-defined

Table 2: Four settlement objects as instances of the tuple, with three for comparison that are not available: the multiparty channel C_k of Section 5, of which C is the case $k = 1$, the leaf T of the LayerTree covenant tree [36], and a native-covenant escrow E'. No covenant or rebinding opcode is active on mainnet at the cited date [8]; BIP 448 [43] is the current proposal for C_k and E', and T needs more than it supplies. MuSig2 is Nick et al. [33].

MIN, 908 in CCV; the owner must be online to build it and to rebuild it after a competing spend, unless a tower holds its EXIT^t authorisation (1,046 vB). The output does not expire; each leaf carries a lease after which its evictor may pay it out on chain, 158 to 203 vB per leaf in a cohort sweep, and keeping a position costs a renewal per period, 151 vB per member in a signed cohort rewrite or 2,616 vB alone. The covenant dependency is one of four opcode tiers: FULL (`OP_CAT`, splice and bit operations, `OP_TX`, CSFS, the EC opcodes), MID (`OP_CAT`, `OP_TX`, CSFS, `OP_TWEAKADD`; four-byte numerics cap a tree at 2^{31} sat), MIN (`OP_CAT`, `OP_TWEAKADD`; exits and splits only, so no vouchers and no channel leaves) and CCV (BIP-443 and `OP_TX`). None is active, three assume an opcode in a tapleaf version its proposal does not define, and BIP 448 supplies the CSFS that MID uses in the version BIP 348 defines and that FULL, and CCV when it hosts vouchers, assume in another; nothing else a tier needs is in BIP 448. Every figure this paper quotes for T is on the draft's model, which prices a whole transaction on a 946 weight-unit skeleton, rounds once, and omits the witness length prefixes (about one percent in FULL and CCV, three in MID and MIN) and, in its baselines, anchors and fee children.

4.2 The extended ledger

Let the state at step n be

$$X_n = (\ell^U, \ell^C, \ell^V, \ell^E,\ h,\ \mathcal{U},\ \mathcal{J},\ \kappa_n),$$

with $\ell^{\cdot}$ the holdings vector for each class, h the pending holds, $\mathcal{U}$ the registry of Section 6, $\mathcal{J}$ the journal, and κ_n the vector of clocks: block height and per-object elapsed time. The per-channel constraint $\ell^C_{i,+} + \ell^C_{i,-} = \chi_i$ is retained for class C only; class V carries a per-holder constraint $\ell^V_u \geq 0$ and the per-server liability of the next subsection; class E carries a per-escrow state.

A cross-class transfer is a settlement event $\sigma = (\text{from}, \text{to}, \text{amount}, \text{clock condition})$, journaled at initiation, held in h, and moving ℓ only when its clock fires: $U \to C$ is a funding transaction with condition "k confirmations"; $V \to C$ is an HTLC-scripted VTXO the server spends into a Lightning HTLC, with condition "HTLC settled"; $V \to U$ is an offboard or a unilateral exit with condition "CSV elapsed"; $U \to E$ is an escrow deposit; $E \to U$ is release after Δ or slash on disprove.

Proposition 2 (cross-class conservation)**.** *Let a finite set of settlement events be journaled and let every event's clock condition have fired or the event have been aborted. Then the sum over classes and holders of ℓ, plus fees paid to relays and miners, plus amounts slashed, equals its initial value plus external funding. During pending intervals a hold h encumbers its source balance without adding an asset. Equivalently, available holdings plus held balances, fees and slashes equal the same total.*

The proof is bookkeeping: each event debits a hold on its source at initiation, and at settlement releases the hold, debits the source, and credits the destination or the slash account; an abort releases the hold and moves nothing. What the proposition rules out is counting a declared cross-system claim as liquidity before its clock has fired, which a ledger without clocks cannot prevent. What it does not establish is that any clock fires: a round may not confirm, an HTLC may time out, a challenge may end in a slash. Those are per-object finality assumptions and are inputs.

4.3 The server bound

Let $V_A(n)$ denote the value held by a cohort A in VTXOs. In this subsection I_A includes every credited receipt or boarding and O_A every settled outgoing payment or offboard, with fees recorded separately. Internal transfers cancel and refresh preserves face value. The holder identity is

$$V_A(n) = V_A(0) + I_A(n) - O_A(n) \geq 0.$$

An issuance financed from another source must appear as an explicit credit in I_A.

The server's advance is a different ledger. Let B_S^F be its fixed reserve available for fronting, $F_S(n)$ cumulative value fronted, and $C_S(n)$ cumulative value recovered. Starting from an empty advance ledger, its unrecovered stock is $W_S(n) = F_S(n) - C_S(n)$. A refresh or a Lightning send can add to F_S; a sweep of expired value adds to C_S [47, 48].

Proposition 3 (pooled fronting constraint)**.** *If a server admits a new advance $f_n \geq 0$ only when*

$$f_n \leq B_S^F - W_S(n^-),$$

and records recoveries only against previously fronted and not yet recovered value, then $0 \leq W_S(n) \leq B_S^F$ at every prefix. This reserve is shared across holders. Cumulative fronting F_S can exceed B_S^F when recovered funds are reused.

Proof. An admitted advance increases W_S by at most the available reserve. A valid recovery reduces it by an amount already included in W_S. Induction over the event sequence proves both inequalities. □

The proposition is a stock constraint at each event. Summing all historical allocations and comparing that total with the current reserve would count reused capital repeatedly. The reference model witnesses reserve reuse explicitly. Its fronting reserve is accounted separately from the stock P_S that supports the holders' aggregate balance; the grid estimates P_S by a forecast $\hat{f}_S$.

Pooling changes the stock forecast from the sum of individual peaks to a peak of aggregate holdings. The gain depends on the demand assumptions in Remark 1. The server also remains a Lightning node, so Lemma 1 constrains its gateway channels. The accounting convention below includes the operator's stock toward holders and excludes the rest of the network's stock toward that operator consistently across objects. A complete deployment comparison must additionally price those gateway channels and identify which holdings and reserves can be reused without double counting.

The server can decline service when its reserve is insufficient. Holders also bear refresh and exit obligations, and pending out-of-round transfers retain the protocol's counterparty assumptions. Those obligations are distinct from the reserve identity.

4.4 The ratio that decides

Definition 2 (liquidity duration)**.** For a demand process D over a horizon H with positive delivered volume, the liquidity duration of an object is locked capital-time per unit delivered,

$$\mathcal{D}_{\text{obj}}(D, H) = \frac{1}{\text{Vol}(D, H)} \int_0^H \text{Locked}_{\text{obj}}(t)\, dt,$$

with Vol counting delivered volume in both directions.

Write Vol_C and Vol_V for the volume each replay actually delivers. If the channel holds $\sum_a \hat{f}_a$ throughout the horizon and the server carries pooled stock $P_S(t)$ in addition to $W_S(t)$, then

$$\frac{\mathcal{D}_C}{\mathcal{D}_V} = \frac{\mathrm{Vol}_V}{\mathrm{Vol}_C} \frac{H \sum_a \hat{f}_a}{\int_0^H [P_S(t) + W_S(t)]\, dt}. \tag{1}$$

The delivered-volume factor equals one only when the two systems serve equal volume. A policy that rejects more demand can change both the numerator and denominator of its duration. An economic comparison therefore also needs a common service target or an explicit penalty for unmet demand.

For advances a_j made at s_j and recovered at r_j, with an empty initial lock ledger,

$$\int_0^H W_S(t)\, dt = \sum_{j:s_j<H} a_j \, [\min(H, r_j) - s_j]_+.$$

This includes advances for refreshes and clips locks at the observation horizon. A rate-times-mean-lock approximation uses the total fronted amount and its amount-weighted lock, which need not equal the bidirectional payment volume. With constant $P_S = \hat{f}_S$ and equal delivered volumes, Equation (1) reduces to a comparison between channel capital-time and pooled stock plus the exact sum of fronting locks.

Current Second documentation states that, after a successful Lightning send, the server retains the HTLC as a safeguard and sweeps the round when it expires [47, 48]. Expiry recovery is therefore the documented case for this implementation. The reference model also carries next-round recovery as a counterfactual sensitivity. That case requires an independently justified recovery mechanism before it can describe a deployed system. Figure 3 illustrates the two assumed clocks.

The implementation also uses different VTXO lifetimes. Its Lightning-receive pool is created with 432-block lifetimes and issues a receive only with at least 144 blocks remaining; received VTXOs therefore expire in one to three nominal days and can be refreshed into standard 4032-block rounds [47]. The current grid uses a single 4032-block lifetime as a stylized model. It establishes the effect of its assumed clock, while a Bark-calibrated comparison must model receive-pool age and refresh separately.

The LayerTree regime. Under T nothing is fronted at a spend. A member pays by voucher, a signature over a leaf that its holder redeems alone, or in a channel leaf with capital of its own, so W_S has no counterpart, and what the operator fronts comes before the receipt. A member receives over Lightning into a credit leaf the operator owns, with the member's key as exit key: the operator settles the payer's HTLC first and then signs the member a cumulative voucher for the running total, and since a voucher draws only what the leaf holds, the operator pre-funds the leaf and tops it up at about 122 vB per leaf in a cohort rewrite. The pre-funded capital P_a is the operator's, bound from top-up until the member co-signs a settlement, and on every path the operator can take alone its excess is the member's, so the forfeit runs the opposite way to Ark's and the operator's incentive is to fund little and top up often; the member's exposure is the one payment settled and not yet vouched for. A pending-leaf variant, with a reclaim key and a landing leaf per member, removes that exposure and bounds the operator's lock at a settle window W of its choosing. Two further terms are bytes rather than capital. A graft admits a cohort at under 3 vB per member when the operator funds it, fronting the cohort's value G until members pay, or nothing if they prepay by voucher, and at 58 vB more per member when

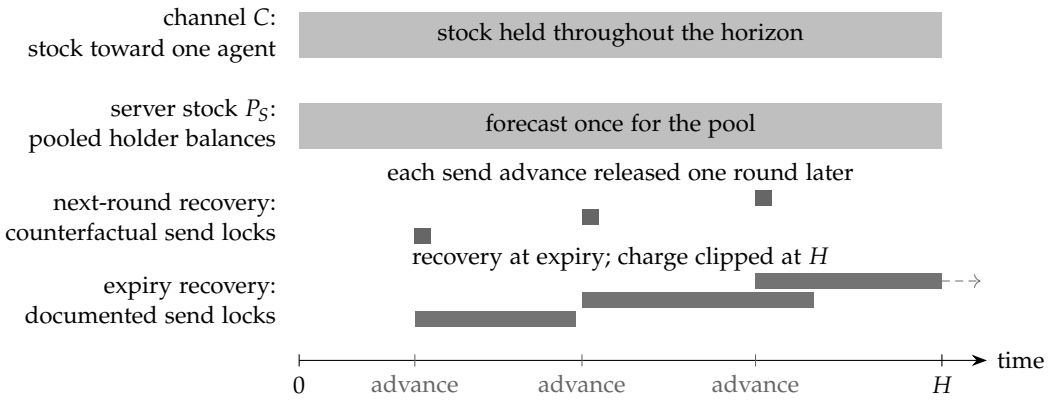


Figure 3: Schematic capital-time accounting. The upper bars show constant provisioning stock; their heights are illustrative. Each lower strip is a separate send advance, so overlapping recovery periods contribute separately. The expiry scenario follows Second's documented recovery after a successful Lightning send. Next-round send recovery is a counterfactual. Refresh advances retain expiry locks in both simulations. A lock contributes its amount times the interval from fronting to recovery, clipped at H; any terminal obligation must also be reported. The model currently counts occupied locks and excludes idle fronting reserve. Table 3 retains both scenarios without assuming a universal mean residual lifetime.

members fund it, which is then a signing round among them; and a lease is renewed at 151 vB per member per period T_ℓ in a cohort, or 2,616 vB alone. With the convention of Section 5.4,

$$\mathrm{Locked}_T(t) = \sum_a P_a(t) + G(t), \qquad P_a \approx (1+m)\, r_a\, T_u, \tag{2}$$

$$\frac{\mathcal{D}_C}{\mathcal{D}_T} \;\approx\; \frac{H \sum_a \hat{f}_a}{H \sum_a (1+m)\, r_a\, T_u \;+\; \int_0^H G\, dt}, \tag{3}$$

where r_a is the member's receipt rate and T_u the top-up period or settle window. This displayed approximation assumes equal delivered volume; otherwise it needs the factor $\mathrm{Vol}_T/\mathrm{Vol}_C$. A separate byte-rate is $b_T = 122/T_u + 151/T_\ell$ vB per member per block in cohorts of 64. The credit leaf funds short-horizon gross receipts per member, while the channel forecast funds a peak net balance. Treating them as comparable requires a specified receipt and spend process.

An indicative calculation uses the corrected zero-drift seed-1 cell: expected receipts of 100 sat per hourly round, a daily top-up, and margin 0.25. It sizes the credit leaf at 3,000 sat per member against channel stock of 105,912.5 sat under the oracle and 38,654.7 sat under the trailing rule. The recurring rate is about 0.88 vB per member per block, or 46,500 vB per nominal year. At 2 sat/vB that is about 93,000 sat annually, against a carry of about 2,058 sat on the oracle capital saving at a 2% annual charge.

We use 2% as a bitcoin-denominated scenario assumption and 0.25% as a sensitivity, motivated by the illustrative node-accrual examples in Bhatia's proposal [6]. Neither value establishes a current market floor. Published advertised liquidity rates [3] require calibration to the operator, term and observation period before use as an empirical cost of capital.

With these assumptions, byte cost equals the carry on saved oracle capital at approximately 4,518, 22,591 and 112,956 sat received per round at 2, 10 and 50 sat/vB. The middle

	Swept at expiry		Next round (counterfactual)	
Demand cell	oracle	trailing	oracle	trailing
200 agents, bursty, drift 0, $H = 8\tau$	0.86	0.47	1.48	1.07
200 agents, bursty, drift 0, $H = 1\tau$	0.65	0.65	1.45	1.45
200 agents, bursty, drift $+60$, $H = 8\tau$	0.97	1.05	0.99	1.11
200 agents, bursty, drift -30, $H = 8\tau$	0.65	0.33	2.59	1.53
3 agents, steady, drift 0, $H = 8\tau$	0.01	0.01	0.85	0.75

Table 3: $\mathcal{D}_C/\mathcal{D}_V$ in the corrected reference model with two-way delivered volume, margin 0.25, a single lifetime $\tau = 4032$ blocks, hourly rounds, and refresh 288 blocks before expiry. Bursty cells draw 5000-sat receipts and 4000-sat spend requests with drift set by their rates; the steady cell draws 100–109 sat in each direction per round. At $H = 1\tau$ the trailing rule coincides with its oracle initialization. Oracle forecasts reject only unaffordable spends. Trailing Ark entries are post hoc sizing diagnostics when the server column in Table 5 is nonzero. Values above one favor V on this occupied-capital metric.

value corresponds to about 1.98 BTC of annual receipts at 24 rounds per day. Using the trailing stock raises the middle value to about 65,207 sat per round; reducing the charge from 2% to 0.25% multiplies every threshold by eight. This is a linear sensitivity calculation. It assumes the chosen capital coefficient continues to scale with receipts, omits deployment-level round and fee-management costs, and does not simulate a credit leaf with the same service as the channel. A receive-only leaf therefore needs its own demand model before these thresholds can guide deployment.

4.5 Reference-model results

The reference model [16] has two forecasts. The oracle provisions the whole-horizon receipt-first peak with a margin and holds that stock throughout. It supplies a benchmark with full future demand information. The trailing rule initializes from the first lifetime's peak, then re-provisions each lifetime from the preceding lifetime; this gives the initial interval an oracle warm start. Subsequent receipts can fail when the prior interval underpredicts them.

The Ark ledger in the grid has an effectively unconstrained fronting reserve. The reported capital-time charges occupied locks and a separately sized stock toward holders; it does not charge idle treasury that a real server might hold against future advances. Its aggregate-flow forecast is evaluated after the holder replay. A failure in that diagnostic does not reduce Ark delivery or change its later locks. Accordingly, a trailing column with server forecast failures is a sizing diagnostic rather than a feasible end-to-end comparison. Even an oracle column remains a stylized occupied-capital calculation under the stated lifetime and refresh rules.

The October 21 correction makes both forecasts and execution process receipts before spends and reject an unaffordable outgoing request in full. It also integrates each lock only until recovery. Table 3 gives the corrected baseline seeds, Table 5 reports their failures, and Table 4 shows the accounting sensitivity. Failure costs and different delivered volumes must accompany any economic reading of these ratios.

The corrected seed-1 zero-drift, eight-lifetime cell has $\mathcal{D}_C/\mathcal{D}_V = 0.86$ under expiry recovery and 1.48 under the next-round counterfactual, using oracle sizing. The draining cell has 0.65 and 2.59. These values show the sensitivity to recovery time within the model.

Accounting, drift +60 cell, expiry lock, oracle forecast	$\mathcal{D}_C/\mathcal{D}_V$
no server-tier term, outbound-only volume	7.15
no server-tier term, two-way volume	25.45
server-tier term charged, two-way volume	0.97

Table 4: The same cell under three accountings. Dropping the server-tier term, or counting only outbound volume on the VTXO side against two-way volume on the channel side, manufactures a large ratio on a cell where all agents accumulate together and pooling has nothing to pool.

		Channel		Server tier	VTXO
Demand cell	events	oracle	trailing	trailing	
200 agents, bursty, drift 0, $H = 8\tau$	48 623	2 766	6 746	24	2 766
200 agents, bursty, drift 0, $H = 1\tau$	6 092	907	907	0	907
200 agents, bursty, drift +60, $H = 8\tau$	32 337	165	6 257	895	165
200 agents, bursty, drift −30, $H = 8\tau$	56 671	8 856	12 713	0	8 856
3 agents, steady, drift 0, $H = 8\tau$	32 256	4	8	0	4

Table 5: Failed events for the corrected baseline seeds. The oracle channel and VTXO holder ledger have the same unaffordable-spend failures. The trailing channel also rejects receipts beyond provisioned inbound; those rejections can change later spends. Server-tier failures are a separate aggregate-flow sizing diagnostic and have not been propagated into the Ark ledger. Recovery mode changes occupied locks, rather than the offered demand or these holder failures.

The few-steady cell also changes materially after the event-order correction: its trailing channel fails 8 of 32,256 events, and its server sizing diagnostic fails none. Earlier claims that this cell mostly refused traffic arose from the forecast bug.

Sensitivity across demand seeds. The October 21 review adds a post hoc campaign with seeds 1–24 in each of four cells. It uses the oracle forecast to compare the same feasible service: in all 96 cell/seed combinations, channels and hyperedges fail only the same unaffordable outgoing requests as the VTXO holder ledger, and the server sizing diagnostic fails none. Table 6 reports means and Monte Carlo intervals. These intervals describe variation in the specified generator, rather than uncertainty about a live network. The campaign and every per-seed value are retained under `audit/peer_review/`.

The expiry case has mean $\mathcal{D}_C/\mathcal{D}_V$ below one in each cell. That result applies to the model's single lifetime and occupied-capital convention; it is not a calibrated claim about Bark's total capital cost. The few-steady counterfactual has mean 0.924 and a realized range from 0.854 to 1.039, so individual demand paths cross the ranking. Disaggregating receive-pool lifetimes, charging treasury reserves, and calibrating budgets to a common holdout service target remain the next quantitative steps. The current model reproduces LayerTree's published byte tables but contains no equivalent-service credit-leaf replay.

4.6 The star topology

If agents hold VTXOs and pay Lightning invoices through the server's HTLC-scripted outputs, the graph the agents see is a star with the server at its center, and the server's

Demand cell	$\mathcal{D}_C/\mathcal{D}_{20}$	$\mathcal{D}_C/\mathcal{D}_N$	$\mathcal{D}_C/\mathcal{D}_V$ expiry	$\mathcal{D}_C/\mathcal{D}_V$ next round
200 agents, zero drift	1.467	1.541	0.871	1.531
	[1.450, 1.483]	[1.522, 1.561]	[0.865, 0.878]	[1.511, 1.551]
200 agents, draining	2.174	2.767	0.649	2.737
	[2.138, 2.211]	[2.725, 2.809]	[0.643, 0.655]	[2.696, 2.778]
200 agents,	1.009	1.009	0.970	0.987
accumulating	[1.008, 1.009]	[1.009, 1.010]	[0.970, 0.970]	[0.987, 0.988]
3 agents, steady	–	1.375	0.007	0.924
		[1.325, 1.424]	[0.007, 0.008]	[0.902, 0.946]

Table 6: Post hoc sensitivity over 24 demand seeds per cell, with equal-service oracle sizing. Each cell shows the mean ratio and its 95% Monte Carlo interval. All lifetimes are 4032 blocks; the next-round send-recovery column is counterfactual, and refreshes retain expiry locks. The intervals concern the specified model and generator. Group size N is 200 in the first three rows and 3 in the last.

own channels are the only edges with direction. The cut deficits live on the server's channel set, not among the agents; the ghost solver's job is to size and place that set given the aggregate demand its holders present, and a ghost edge compiles to a connector on the server side. The apparatus of Section 3 is moved one tier up and applied to a smaller, better-observed graph, with $\hat{f}_S$ as its target. A LayerTree is a star of the same kind for Lightning receipts, which arrive on the operator's channels and are vouched into credit leaves; a voucher between two members crosses no operator and no cut the operator sees until it is redeemed. The separate repository note `paper/designed_topology_note.tex` preserves an exploratory discussion of designed overlays; none of the capital comparisons here uses it.

4.7 One output: throughput as a constraint

Every LayerTree transition spends the current tree output and recreates its successor. Competing transitions must therefore be ordered, and users may have to reconstruct a transaction after an intervening transition. The draft collects this issue in a contention assumption [36]. A throughput ceiling by itself cannot bound waiting time: a miner can confirm fewer transactions than the ceiling allows.

For a conditional queueing calculation, suppose each service interval of Δ_L blocks clears at least the smaller of κ and the current eligible backlog while the honest request remains pending. Suppose also that at most β adversarial transitions overtake it and that the honest user promptly reconstructs and relays a valid successor spend. If q requests are initially ahead of it, a conservative delay bound is

$$\Delta(q) \leq \Delta_L \left(\left\lceil \frac{\beta + q}{\kappa} \right\rceil + 1 \right). \tag{4}$$

The service, bounded-overtaking and resubmission premises must all be supplied. Bitcoin's mempool ancestor, descendant and cluster admission limits establish none of them and are not consensus limits on the number of transitions confirmed in a block. Values $\kappa = 1, 25, 64$ are illustrative service scenarios in the reference model. At a constant service of 25 or 64 transitions per block, 10,000 cooperating exits require 400 or 157 service blocks, respectively, before adding initial inclusion delay or interference. These figures are queue scenarios, rather than deployment guarantees.

Independent channel outputs can be spent concurrently, subject to shared blockspace. An Ark exit progressively exposes branch outputs, and a multiparty channel requires the state-update and settlement sequence specified by its protocol. Those transaction graphs should be measured separately instead of assigning each the LayerTree recurrence. LayerTree's detach operation can create independent output queues for aligned cohorts; the draft gives 780 vB for a cohort of 256 at depth 16.

Under the same chosen service model, a budget calculation can price repeated adversarial transitions. A model-size carve of 1,586 vB repeated κ times costs $1{,}586\kappa f$ sat at feerate f. This describes expenditure conditional on those transactions being admitted and confirmed. It does not prove that a given expenditure delays an honest exit, or that the honest bid defeats pinning. A controlled regtest experiment can measure accepted conflicts, reconstruction and service under a specified miner policy. Any statistic on the public spend chain must also justify which leaf activity is observable under the selected opcode tier.

5 BIP 448: the bound under rebindable transactions

The Lightning accounting identity applies to deployed channels. This section analyzes a proposed soft fork that changes how a shared output can update its state. BIP 448 [43] is a draft, assigned in March 2026 and not activated, that adds three operations to Tapscript. `OP_TEMPLATEHASH` [42] pushes a hash of the spending transaction's version, locktime, input sequences, outputs, the current input index and annex-presence flag, so that an output can commit to the transaction that must spend it without committing to which coin funds it. `OP_CHECKSIGFROMSTACK`, abbreviated CSFS below, verifies a signature on a stack message, and `OP_INTERNALKEY` pushes the Taproot internal key. Together they give signatures that are rebindable: valid for any output under the same key, not for one outpoint. The construction they are meant for is LN-Symmetry [9, 14], in which any later channel state can spend any earlier one, so that there is no revoked state to punish and nothing asymmetric for each party to hold. Two parties get a simpler channel from that. The change that matters here is to the number of parties. A penalty needs someone to blame and a state that one party holds against one other, and extends poorly past two; a symmetric state signed by all members extends to any number, and the proposal's own motivation is that this "makes advanced constructs like multiparty channels practical." We take that object as given and ask what the bound becomes.

5.1 From edges to hyperedges

Let the channel set E of Section 3 be a set of hyperedges. Channel i has a member set $\mathcal{M}_i \subseteq N$ with $|\mathcal{M}_i| \geq 2$, a capacity χ_i, and balances $\ell_i(j) \geq 0$ for $j \in \mathcal{M}_i$ with $\sum_{j\in\mathcal{M}_i} \ell_i(j) = \chi_i$. A member spends only its own balance: a transfer of a from u to w inside channel i is feasible when $\ell_i(u) \geq a$, and is a state update that every member of $\mathcal{M}_i$ signs. The two-party channel is the case $|\mathcal{M}_i| = 2$. For $A \subseteq N$ let ∂A be the channels with members on both sides of the cut, let $S_A(n) = \sum_{i\in\partial A} \sum_{j\in\mathcal{M}_i\cap A} \ell_i(j)$ and $S'_A(n) = \sum_{i\in\partial A} \chi_i - S_A(n)$, and let O_A, I_A, K_A, K'_A be as before.

Lemma 2 (hyperedge cut bound)**.** *On the hypergraph the inequalities of Lemma 1 hold for every $A \subseteq N$ and every n, under every routing policy.*

Proof. A transfer inside a channel $i \in \partial A$ either moves its amount between a member in A and a member outside, changing S_A and S'_A by that amount in opposite directions, or

moves it between two members on the same side, changing neither. Channels outside ∂A are not in the sums. So $S_A(n) = S_A(0) - O_A(n) + I_A(n) + K_A(n) \geq 0$ as before, and likewise for S'_A; nothing in the argument depends on how attempts are chosen. □

The soft fork does not relax the bound. Conservation and nonnegativity belong to any object that holds value as balances, and Proposition 1 carries over word for word. What changes is how finely capacity is partitioned, which is what the provisioning problem of Section 3 was about. Figure 1(b) draws the same cut through a hyperedge.

Proposition 4 (pooling inside a hyperedge)**.** *Let channel i have members $\mathcal{M}_i = \{h\} \cup A_i$: a hub h through which the agents A_i reach the rest of the network, with $|A_i| = k$. Then (i) the inbound stock toward the agents on i is the single number $\ell_i(h)$, and net receipts into A_i over i are bounded by $\ell_i(h)$ at time zero plus what capacity events add, with no separate constraint on any one agent's receipts; (ii) a payment between two agents of A_i consumes none of $\ell_i(h)$; and (iii) the hub's provisioning problem on i is one newsvendor problem for the peak of the total its k agents hold, $\hat{f}_{A_i}$, in place of k problems for the $\hat{f}_a$.*

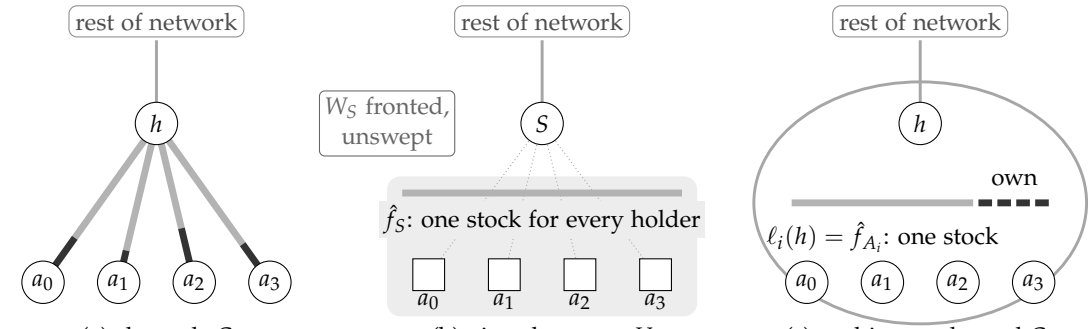
Proof. (i) is Lemma 2 on the cut A_i, where $S'_{A_i} = \ell_i(h)$ on channel i. For a single agent a the inbound on i is $\chi_i - \ell_i(a) = \ell_i(h) + \sum_{b \in A_i \setminus a} \ell_i(b) \geq \ell_i(h)$, so a receipt through the hub that the cut A_i admits, the cut $\{a\}$ admits. (ii) is the second case in the lemma's proof. (iii) follows from (i): the stock that has to be placed before demand is known is $\ell_i(h)$, and what exhausts it is the total held by A_i. □

The two-party channel is $k = 1$, and there the division of χ_i between the hub's part and the agent's part is the direction partition of Section 3. With k agents the division has $k+1$ parts, but only the hub's is a stock that someone had to place by forecast; the agents' parts are their own money. Direction, which was a property of an edge, becomes a matter of who holds what inside a pool, and the pool reassigns it by signature. That is what Section 4 said of the virtual output, and here it is said without a server. Figure 4 puts the three objects side by side. The LayerTree leaf is a third form of the same statement: no direction at the member tier, since a leaf is paid to anyone by voucher, one output for all, and an operator needed to graft and to co-sign an update but not to pay, to redeem or to exit.

With N agents in N/k hyperedges the hub carries $\sum_i \hat{f}_{A_i}$. Remark 1 applies inside each group with k in place of N: independent signed imbalances give an order-$\sqrt{k}$ benchmark, while the zero-start reflected-Brownian ratio of expectations approaches $\pi/2$. Neither is a sample-path guarantee for the finite spend-rejection model. Shared accumulating drift can dominate the benefit. A population on per-agent channels is a multi-location newsvendor problem; pooling can lower the stock required under the demand and cost assumptions studied by Eppen [18]. The hyperedge provides that pool at the scale its members can sustain.

5.2 The clock on capacity

Changing a channel's total capacity can require an on-chain funding, closing or splice transaction. Rebalancing its directional shares can also occur through off-chain payments; such movements enter the cut ledger as settled crossings and require feasible paths. A hyperedge has a second kind of capacity event. In the factory construction [10] the shared output funds a set of two-party subchannels, payments run in the subchannels at the speed and concurrency of ordinary channels, and the members re-divide the shared output



(a) channels C
k stocks, one per edge: $\sum_a \hat{f}_a$

(b) virtual outputs V
one stock, $\hat{f}_S$, plus W_S

(c) multiparty channel C_k
one stock, $\hat{f}_{A_i}$, nothing fronted

Figure 4: Where the forecast lives, for the same k agents and a hub. Light segments are stock placed by forecast before demand is known; dark segments are the agents' own balances. (a) Per-agent channels: the hub holds a separate stock on every edge, and the total is a sum of k forecast peaks. (b) An Ark: outputs have no direction at the holder tier, the server co-signs every transfer, its stock is forecast once for all holders, and value fronted at spends and refreshes waits unswept as W_S (Proposition 3). (c) A multiparty channel: one shared output whose hub part is the single stock for the group, forecast once, with nothing fronted and every update signed by all $k+1$ members (Proposition 4). The gateway channel from h or S is drawn without a stock. Its capital is excluded by the stated operator-stock convention; equal gateway cost across mechanisms is an assumption requiring verification.

among the subchannels by signing a new allocation, with no transaction on chain. Write $R_a(n)$ for the net capacity moved toward agent a by such reallocations inside a factory with member set $\mathcal{M}$. The bound on a's net receipts becomes

$$I_a(n) - O_a(n) \;\le\; S'_a(0) + K'_a(n) + R_a(n), \qquad \sum_{a \in \mathcal{M}} R_a(n) = 0, \tag{5}$$

where K'_a still carries a confirmation clock and R_a carries the latency of a signing round among $\mathcal{M}$. A per-agent forecast that turns out wrong is corrected from another agent's idle capacity in the time it takes $|\mathcal{M}|$ parties to sign, and only the factory's total remains a forecast with a chain clock. A factory that reallocates every round has the bound of the single multiparty channel of Proposition 4, and one that never reallocates has the bound of k separate channels. The demand-shift row of Table 1 shows the tested placement heuristic losing its stationary advantage; inside a factory it can be, among the members and no further. Figure 5 draws the two clocks. LayerTree's cohort rewrite is the factory's reallocation with a transaction for a clock, 151 vB per changed leaf in a cohort of 64 with only the changed owners and the operator signing, so a per-leaf forecast is corrected there on chain at that price rather than off chain for a signature.

5.3 What the pooling costs

The hyperedge buys its pooling with interactivity. Every update of a multiparty channel, and every reallocation of a factory, needs a signature from every member, so the object's liveness obligation is the conjunction of its members' and its counterparty set Γ is every co-member: not for custody, since no state exists that its holder did not sign, but for availability, since one member that stops signing freezes the pool at its last state and leaves

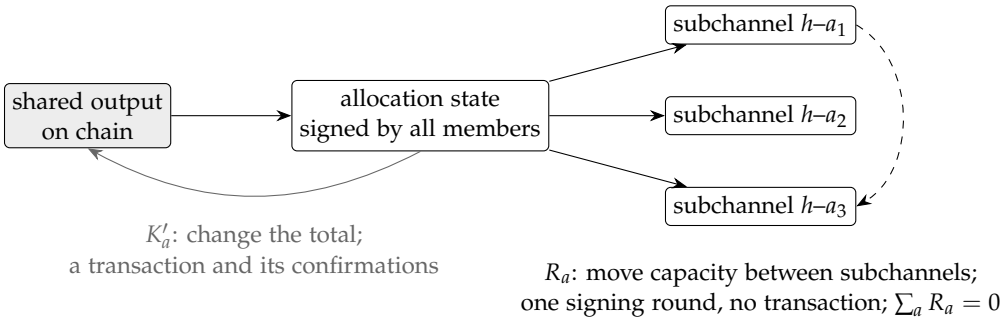


Figure 5: A factory's two clocks. The shared output is a chain object; the allocation that divides it among the subchannels is an off-chain state that every member signs. Moving capacity between subchannels, R_a in Equation (5), takes one signing round and conserves the total; changing the total, K'_a, takes a transaction on chain and its confirmations. Inside the factory a forecast made per agent can be corrected from another agent's idle capacity at the speed of a signature, and only the factory's total remains a chain-clocked forecast.

the others to exit on chain. An exit publishes the latest state and waits out a relative timelock before it settles, and the state divides the output among all $k+1$ members, so one member's failure costs chain space of order k unless the state is itself a tree of templates, in which case an exit reveals a path and the pool survives only in the subtrees beside it. A covenant tree of the LayerTree kind is the other case: one member leaves in one transaction of $O(D)$ bytes and the output survives, recreated under the new root, for the rest. If each member independently fails or defects within the horizon with probability p, the pool survives it with probability $(1-p)^{k+1}$, which holds k to the order of $1/p$, while by Remark 1 most of what a pool of agents with nothing to spend down can gain has arrived by a k of a few tens.

Turnover also exposes a hyperedge to shared exit costs. In the assumed multiparty construction, changing membership respends the funding output and requires the existing members to sign. A cooperative departure costs a transaction; an uncooperative departure can force the group to publish and settle its state, with order-k output cost. For an illustrative hazard model, let the $k+1$ members depart as independent Poisson processes of rate λ, and independently let each departure be uncooperative with probability q. Then a member has uncooperative-departure probability $p = 1 - \exp(-\lambda q T)$ over T, and the expected time to the first such departure is $1/((k+1)\lambda q)$. Multiplying this hazard by the group settlement cost gives order-$k\lambda q$ output cost per agent per unit time before failure. Interpreting it as a long-run rate additionally assumes immediate reconstitution of fixed-size pools. Per-agent channels isolate the departing member's close. Ark and LayerTree can also avoid forcing every co-member to exit, but their marginal sweep bytes, refresh duties, recovery locks and lease expirations still carry costs. Choosing k requires pricing those obligations together with the measured pooling benefit; the model has not optimized that choice for a changing population.

Section 4 observed that liveness distinguishes agents from humans and not Ark from channels. It does so more sharply here, because the hyperedge asks every member to be reachable for every other member's reallocation, which a population of always-online processes can supply and a population of phones cannot. The same proposal lowers the liveness price of the alternative: Ark variants built on these operations run rounds with no synchronous participation and let a server refresh an absent holder's output indefinitely

	$\mathcal{D}_C/\mathcal{D}_k$, oracle			$\mathcal{D}_C/\mathcal{D}_k$, trailing			$\mathcal{D}_V/\mathcal{D}_N$	
Demand cell	$k=5$	$k=20$	$k=N$	$k=5$	$k=20$	$k=N$	expiry	round
200 agents, bursty, drift 0, $H=8\tau$	1.29	1.43	1.49	0.99	1.02	1.03	1.73	1.01
200 agents, bursty, drift 0, $H=1\tau$	1.28	1.43	1.46	1.28	1.43	1.46	2.25	1.01
200 agents, bursty, drift $+60$, $H=8\tau$	1.01	1.01	1.01	1.01	1.01	1.02	1.04	1.02
200 agents, bursty, drift -30, $H=8\tau$	1.61	2.14	2.62	1.21	1.38	1.55	4.04	1.01
3 agents, steady, drift 0, $H=8\tau$	–	–	1.19	–	–	1.25	156.87	1.40

Table 7: Multiparty channels of k agents and a hub on the demand of Table 3, with the same margin and forecast rules. $\mathcal{D}_C/\mathcal{D}_k$ above one favors the hyperedge over per-agent channels. $\mathcal{D}_V/\mathcal{D}_N$ is at $k=N$ under the oracle forecast, for each recovery scenario; above one favors the hyperedge over the virtual output. The few-agent cell has only $k=N=3$. Generated by `model/grid.ts`.

[40]. That removes the obligation Λ of the V row for humans and changes little for agents, who met it already; and neither variant is described as changing what the server fronts or for how long, so W_S and the recovery question of Section 4 stand as they were. BIP 448 therefore narrows the advantage agents had over humans under Ark and opens a different one under the hyperedge.

5.4 The ratio with three objects

Write $\mathcal{D}_k$ for hyperedges of k agents and a hub, so that $\mathcal{D}_1 = \mathcal{D}_C$ under the same event order and forecast. For equal delivered volume, constant stocks over H, and the fronting integral of Section 4,

$$\frac{\mathcal{D}_C}{\mathcal{D}_k} = \frac{\sum_a \hat{f}_a}{\sum_i \hat{f}_{A_i}}, \qquad \frac{\mathcal{D}_V}{\mathcal{D}_k} = \frac{H^{-1}\int_0^H W_S(t)\,dt + \hat{f}_S}{\sum_i \hat{f}_{A_i}}. \tag{6}$$

Unequal delivered volumes require the corresponding volume ratios from Equation (1). The operator-stock convention excludes the rest of the network's capital toward the operator. An identical added gateway cost moves equal-service ratios toward one without changing their ordering; the identical-cost premise needs verification when mechanisms change traffic or settlement timing. The aggregate server diagnostic also groups receipts before spends within each round, so transient peaks can differ from a hyperedge replay that processes members sequentially. The tables compute both explicitly rather than asserting that the two stocks coincide exactly.

5.5 Reference-model results

The reference model groups the agents of each cell k to a hyperedge and replays the demand of Table 3 against it. A receipt succeeds when the hub's balance in the group's channel covers it and a spend when the agent's own balance does, and the hub's stock is set by the same two forecast rules, applied to the total the group holds. At $k=1$ the computation is the channel model's, operation for operation, which the generator checks before it prints. Table 7 gives computed duration ratios, including actual delivered-volume factors when service differs, and Table 8 gives failed events under the trailing rule. Equation (6) supplies the equal-volume special case.

On the corrected baseline zero-drift cell over eight lifetimes, the oracle ratio $\mathcal{D}_C/\mathcal{D}_k$ is 1.29 at $k=5$, 1.43 at $k=20$, and 1.49 at $k=200$. With trailing forecasts, failed events fall

		Hyperedge, trailing forecast				VTXO
Demand cell	events	$k = 1$	$k = 5$	$k = 20$	$k = N$	
200 agents, bursty, drift 0, $H = 8\tau$	48 623	6 746	4 142	3 393	3 116	2 766
200 agents, bursty, drift 0, $H = 1\tau$	6 092	907	907	907	907	907
200 agents, bursty, drift $+60$, $H = 8\tau$	32 337	6 257	5 071	4 918	4 826	165
200 agents, bursty, drift -30, $H = 8\tau$	56 671	12 713	9 847	9 037	8 856	8 856
3 agents, steady, drift 0, $H = 8\tau$	32 256	8	–	–	5	4

Table 8: Failed events under the trailing forecast as the group grows, out of the events each cell attempts. $k = 1$ is the per-agent channel of Table 5, and the VTXO column counts unaffordable outgoing requests in the unconstrained holder replay; forecast-induced receipt failures can change later affordability in the other columns. Generated by `model/grid.ts`.

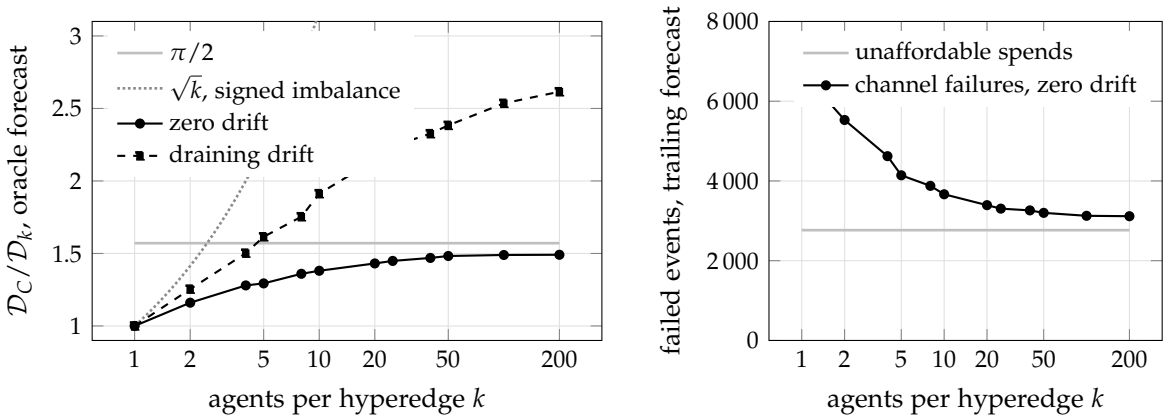


Figure 6: Left: one baseline realization of the multiparty-channel pooling gain under oracle sizing, for the zero-drift and draining cells of Table 7. The $\pi/2$ line is the reflected-Brownian ratio-of-expectations benchmark, not a cap on individual runs of this finite spend-rejection model (Remark 1). The signed-imbalance $\sqrt{k}$ benchmark leaves the plot by $k = 10$. Right: zero-drift failed events under the trailing forecast for this realization. The horizontal reference counts unaffordable outgoing requests in the unconstrained holder replay. Forecast-induced receipt failures can change later affordability. Generated by `model/grid.ts`; Table 6 reports the separate multi-seed sensitivity.

from 6,746 at $k = 1$ to 4,142, 3,393 and 3,116 at those group sizes, against 2,766 unaffordable outgoing requests in the unconstrained holder replay. These are complementary views of stock cost and delivered service.

The sensitivity campaign in Table 6 gives a zero-drift mean ratio of 1.467 at $k = 20$ and 1.541 at $k = 200$. Twenty-agent groups capture 86.3% of the excess ratio gain of the full pool, calculated per seed as $(\mathcal{D}_C/\mathcal{D}_{20} - 1)/(\mathcal{D}_C/\mathcal{D}_{200} - 1)$ and then averaged. Under draining drift the corresponding fraction is 66.5%. Thus the useful group size depends on the demand process. A full-pool realized ratio reaches 1.627 on one zero-drift seed, consistent with treating $\pi/2$ as the Brownian ratio-of-expectations benchmark rather than a cap on each finite jump-process run.

The ratios include no membership turnover, signing latency or chain fees. They also contain no payments between members of one group; such payments can change the traffic served by each mechanism and need their own workload. The stock comparison alone therefore does not select an operational architecture or a group size.

5.6 Keys alone, rebinding, and covenants

Three mechanisms put several parties behind one output, and this paper has used two of them. They are worth setting side by side on the terms the bound prices, because they differ in what must be signed and by whom, which is the liveness column of Table 2 and the turnover cost of Section 5, and in what an observer can tell, which is what Section 7 prices.

Keys alone. MuSig2 [33, 34] makes n keys one key: the output is a plain BIP340 key, the signature a plain signature, and nothing in consensus changes. Its nested form, proposed as a draft BIP [19], lets any participant of an aggregation be itself an aggregate, so that a tree of signing groups produces one signature under one root key, and neither the verifier nor a co-signer outside a subgroup learns whether a key is one signer or a group. Both are n-of-n at every node; nesting adds leaf signers to a signature and never removes one. What a key alone does not give is a state. A shared output held by a key is spent by whatever its holders have signed, so a multiparty position built this way is a set of pre-signed transactions: the factory of Section 5 without symmetry, in which every state change is signed by all, a stale state has to be guarded by a penalty or a window, and a member leaves by the pre-signed tree, $O(\log N)$ transactions. The capital bound is the hyperedge's, since the stock is one per shared output whatever its key is made of; the liveness obligation is the hyperedge's, with every leaf signer the nesting adds.

Rebinding. BIP 448 [43] keeps the keys and changes what a signature binds to: the latest state spends any earlier one. The multiparty channel of Section 5 is the result, still n-of-n at each update and $O(k)$ bytes on chain when one member leaves without the others, with the pooling, the clocks and the turnover cost that section gives.

A covenant tree. A third mechanism takes the co-signers out of the exit altogether. LayerTree [36], a draft dated October 2026, commits a Merkle-sum tree of balances to one output and lets a member leave in one transaction of $O(D)$ bytes, about 1,020 vB at depth 16 on the draft's own transaction model, on its signature alone, with an operator that grafts members and co-signs updates but cannot delay an exit; the price is a larger set of opcodes than BIP 448's, in four tiers none of which is active. Table 2 carries it as T, Section 4 recomputes the bound under its lease, graft and credit-leaf terms, and Section 4.7 gives the throughput constraint it brings.

What nesting changes for this paper. Nothing in the bound and something in the observer model. The stock is per object and does not care how the object's key is built. But a member's key may be a group: a principal can be a required co-signer inside every agent's key, in a channel, a hyperedge or a leaf, with no script and with no co-member the wiser, which is the lever of Section 7.5 at the key level; and for the same reason a swarm can be one controller behind many apparently distinct members, and nothing on chain or in the state says so. We reviewed the nested draft and posted findings on its pull request (October 9, 2026): the nonce-binding hash input was defined three ways, the theorems it cites are stated for a depth of at most two where the draft allows any, and its BIP 327 vectors were regenerated for sorted key lists. Its author has since answered the first (October 15, 2026); the rest we have not re-checked. Nothing here depends on the draft beyond its premise, that a nested aggregate verifies as a plain key.

	keys alone: MuSig2, nested	rebinding: BIP 448	covenant tree: LayerTree
needs today	nothing	a soft fork	a larger soft fork, four tiers
an update is signed by	all	all	owner and operator; a voucher by its issuer alone
an exit is	the pre-signed tree, $O(\log N)$ transactions	latest state and settlement, $O(k)$ bytes	one transaction, $O(D)$ bytes, owner alone
a stale state is	penalised, or waited out	replaced	absent
stock placed by forecast	one per shared output	one per hyperedge	one per credit leaf, per member
who sees the structure	nobody, from the key; nesting hides a group inside a member	every member, from the state	the operator, and every member following the output

Table 9: Three ways to put several parties behind one output, on the terms this paper prices. Keys alone is MuSig2 [34] and its nested draft [19]; the covenant tree is LayerTree [36], measured on its own transaction model.

The bound by tier. Table 9 orders the mechanisms by lineage. Ordered by what each asks of script instead, the objects of Table 2 form a ladder, and the bound is piecewise on it: at a rung an operator has the objects that rung admits and can compare their forecast stock requirements at a common service target, with the terms Sections 4 and 5 derived for each. Table 10 gives the rungs. Three things follow from reading the objects this way. The forfeit lock of equation (1) is not a property of pooled settlement but of pooling in an object that is re-funded on chain each round: a template hash alone moves the Ark exit tree from the server's signature to a covenant and leaves the round, its expiry and the lock where they were, as the draft's own comparison lists them [36, Table 12], and the lock leaves the bound at the first rung that admits an object whose balances change without the output being re-funded, which is BIP 448's for a hyperedge and, for a tree, the rung at which the tree becomes a payment object. The lowest LayerTree rung is not yet one in this paper's sense: MIN compiles the exit and the split, the one member-to-member transfer it supports, and both are chain transactions, so nothing moves off chain and no forecast term arises; channel leaves begin at MID or CCV, and vouchers, and with them the credit leaf of Section 4, at MID, or at CCV with CSFS; both tiers assume `OP_TX` in a tapleaf version its draft does not define, and FULL adds the smaller exit and the payout that pays its own fee. Which opcodes it needs is, by the draft's own count, a different set at each rung, and the rung that needs least gives up the vouchers and channel leaves that make the tree a payment object.

6 Proof economics: a reward layer without identities

Whoever provisions the capital that Lemma 1 makes scarce needs to be paid for it, and a population without identities needs a way to pay that does not depend on knowing who is who. This section gives the mechanism and then the cost of running it on a chain that verifies no proofs itself.

script has	objects	stock the forecast places	what the rung changes
BIP 342 as deployed	U, C, V, E	per edge, $\sum_a \hat{f}_a$; or the server's $\hat{f}_S$ plus the forfeit lock $H^{-1} \int W_S(t)\,dt$	the baseline: Lemma 1 and equation (1)
+ `OP_TEMPLATEHASH` (BIP 446)	the same four	the same	the Ark exit tree is a covenant and not the server's signature; the round and its expiry stay
+ BIP 448 (446, 348, 349)	adds C_k	$\sum_i \hat{f}_{A_i}$, pooled in each group, no lock; the pool forced on chain at rate $(k+1)\lambda q$	the server and the forfeit lock leave the bound; every member signs every update; its CSFS is MID's as defined, not the `0xC2` form FULL and CCV assume
+ `OP_CAT` (347) and `OP_TWEAKADD` (449), current leaf version	adds T at MIN	none: no transfer off chain, no graft	a shared output with a unilateral exit, 1,473 vB; no vouchers, no channel leaves
+ `OP_TX` and CSFS in `0xC0` (MID), or `OP_CCV` (443) with `OP_TX`	adds T with channel leaves, and vouchers (under CCV only with CSFS)	the operator's credit leaves and grafts, $\sum_a P_a + G$, and the byte-rate b_T; without vouchers no credit leaf, grafts alone	equation (3) and the constraint (4); 1,355 or 908 vB an exit
+ splice, bit and EC opcodes in `0xC2` (FULL)	the same	the same	1,020 vB an exit, 921 when the payout pays its fee

Table 10: The objects by what script has, and the stock a forecast places at each rung. The rungs are the drafts' as of October 2026 and form a ladder with one fork in it: BIP 448 and the `OP_CAT` tiers are independent proposals, and BIP 448's CSFS is what MID uses where BIP 348 defines it and what FULL, and CCV for vouchers, assume in `0xC2`; nothing else a tier needs is in BIP 448 (Section 4.1). Exit sizes are on the LayerTree draft's transaction model at $D = 16$.

6.1 Why weights have to be authenticated

Lemma 3 (identity splitting)**.** *Let a fixed reward pool P be allocated to claimants in proportion to integer weights w_u attached to scarce resources u. If each w_u is the value of a distinct settled object that a claimant controls, then splitting one claimant into m claimants whose weights sum to w_u leaves the coalition's payout invariant up to rounding. If weights are self-reported, a claimant with one object of value w can report it under m identities and receive m shares.*

The lemma is obvious and is stated because the second half is what an implementation does by default. A registry that records weights without authenticating that distinct claims are backed by distinct objects allocates to identities, and identities are free [15]. Authentication has two routes: the allocator learns who owns what, which defeats the purpose of a permissionless service market; or a claimant proves it controls a settled object of at least the claimed value and that no other claim uses the same object, without revealing which object. The second is the construction below.

6.2 The unique-exposure relation

Let $\mathcal{S}_n$ be an authenticated eligible set at step n. Each object has a unique identifier id_ρ, a positive integer value v_ρ, and one canonical single-signer credential k_ρ. Eligibility must exclude alternative credentials for the same object and overlapping claims on the same

backing: a funding UTXO and the channel balance it backs cannot both enter as independent capital. A public channel-capacity record alone does not authenticate a holder's directional balance.

Let cm commit to a claimant secret s. For an admitted single-signer object define

$$\mathcal{R}_{\text{cap}} = \Big\{ \big((\text{cm}, v, \text{nf}, \text{root}_n),\ (s, k_\rho, \rho, \pi_\rho)\big) : \text{cm} = \text{Com}(s), \quad \rho \in \mathcal{S}_n \text{ via } \pi_\rho, \\ k_\rho \text{ is the canonical credential for } \rho, \\ v \in \mathbb{Z}_{>0}, \quad v \le v_\rho, \\ \text{nf} = \text{PRF}_{k_\rho}(\text{id}_\rho) \Big\}.$$

The registry accepts a proof against its authenticated root only when the nullifier is unused. It then records $(\text{cm}, v, \text{nf})$. A nullifier keyed by the claimant secret would permit another claim after changing identities.

Proposition 5 (claim-once weight)**.** *Assume a knowledge-sound proof system, an authenticated eligible set with unique identifiers and one canonical nullifier credential per object, and a registry that rejects repeated nullifiers. Then the accepted weight backed by any one eligible object is at most its value, except with negligible probability. Summing this result bounds eligible capital only when the admitted objects represent disjoint exposure.*

Proof. Every valid claim on a fixed object derives the same nullifier. The registry accepts at most one such claim and the relation bounds its positive weight by v_ρ. Knowledge soundness excludes an accepted proof of a false relation except with negligible probability. □

Zero knowledge hides the witness beyond the public statement and the system's side information. The public value, root, submission metadata and stable nullifier can still support inference or linking across epochs. The argument follows the claim-once use of a Zerocash-style nullifier [4]; pseudorandomness concerns privacy, while uniqueness requires a canonical credential. A threshold or multipath authority can admit several valid spending witnesses. Deriving a secret-key PRF from an aggregate public key does not resolve that case; a separate distributed nullifier protocol or registry-assigned credential is required. The present reference verifier models the single-credential case only.

6.3 What the transplant leaves open

A nullifier stops a second claim on the same identifier. It says nothing about a claim on a successor identifier, and spending is the one thing a nullifier is designed to permit. Spend ρ to a fresh ρ' under a key the same controller holds, by a self-transfer on-chain or in a round, and $\text{PRF}_{k_{\rho'}}(\text{id}_{\rho'})$ is fresh, so Proposition 5 is satisfied per object while one unit of capital is counted twice. Under Ark the case is structural rather than adversarial: every refresh rotates the identifier, at a time the holder chooses.

The registry therefore takes claims by epoch. A claim is accepted only against $\mathcal{S}$ as frozen at epoch start, so a successor created mid-epoch is not claimable until the next; at epoch close a claim is revoked if its object is no longer settled, unless the set's maintainer has attested a successor link, which for class V is the server's record that a refresh reissued ρ as ρ' under the same key, and which a transfer to another key never receives.

Proposition 6 (epoch bound)**.** *Under the epoch rule and the assumptions of Proposition 5, the weight attributed in one epoch to any lineage of objects under one authority is at most the value*

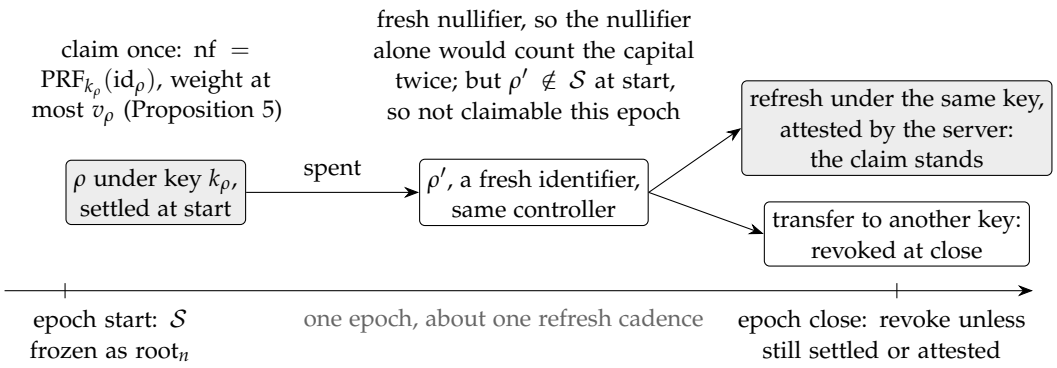


Figure 7: The epoch rule (Proposition 6). A nullifier keyed by the object's authority stops a second claim on the same identifier, but spending is what a nullifier permits, and a spend to a fresh identifier under the same controller carries a fresh nullifier. Claims are therefore accepted only against the settled set frozen at epoch start, so a successor made mid-epoch waits for the next; and at epoch close a claim whose object is no longer settled is revoked unless the set's maintainer attests a same-key successor, which an Ark refresh receives and a transfer to another key does not. Without the rule the reference model accepts eight claims on one object spent to eight identifiers.

settled under that authority at epoch start, whatever the number of identifiers the lineage passes through within the epoch.

The epoch length should not much exceed the refresh cadence, or an honest holder waits a lifetime to claim a boarded output; and the successor attestation is one more statement the verifier trusts the server for, added to the co-signing trust it already carries. The reference model witnesses both propositions on finite cases and reproduces the hole: a registry without the epoch rule accepts eight claims on one object spent to eight identifiers. Figure 7 draws the rule and the hole it closes.

6.4 The service relation

A reward for service needs an authenticated count of eligible requests. Suppose a connector submits m signed outcomes and proves that k say "honored." The ratio k/m then describes the submitted records. A connector that omits failed requests or repeats successful ones can change it without improving service.

A prospective relation therefore needs a committed request ledger. Let L bind the unique request identifiers admitted during $[t_0, t_1]$, their acceptance times and response deadlines, under a specified admission policy. Outcome evidence must cover each entry exactly once; an unfulfilled request at its deadline counts as a failure. A service relation would verify the commitment to L, the admissibility and uniqueness of its $m > 0$ requests, every outcome or timeout, and at least k fulfilled entries. Counterparty attestations must bind the connector, request identifier, window and outcome. The public statement includes the ledger root and the denominator m.

Call that prospective relation $\mathcal{R}_{\mathrm{svc}}$. A proof of it can establish the fraction fulfilled among the admitted requests. Claims about availability to arbitrary callers additionally require a way to observe requests the connector refused to admit. Valid counterparty keys also do not establish independent control, so collusive attestations remain a separate problem. The repository has neither this admission protocol nor a real circuit for $\mathcal{R}_{\mathrm{svc}}$;

the escrow state machine assumes an external verdict and does not validate an end-to-end service metric.

6.5 Who verifies, and what it costs

Bitcoin consensus verifies no succinct argument. The verifier of a claim on $\mathcal{R}_{\text{cap}}$ or $\mathcal{R}_{\text{svc}}$ is therefore one of three parties: the reward allocator, a trusted server; the Ark server, already trusted for co-signing and well placed to hold root_n for class V; or an optimistic game in which an operator asserts the statement and any challenger can disprove it on-chain.

Prover cost depends on the implemented relation, Merkle depth, hash function and proof-system parameters. Neither relation has been compiled or benchmarked here. The implementation experiment will report constraint count, proving-key size, peak memory and elapsed proving time on named hardware. Groth16 [21] specifies a proof construction; it does not establish application-specific timings. For verification we use one concrete published backend: the Yao/Groth16 comparison in BitVM3 [32]: an Assert of about 2.4 kvB, a Disprove of about 93 vB, an off-chain garbled circuit of about 41 GB for a Groth16 verifier that takes about six minutes to garble and six to evaluate, seven such circuits (288 GB) and some three and a half hours of setup when the garbling is checked by cut-and-choose, one honest challenger online throughout, and a committee honest at setup with one signer deleting its key afterward. Soundness is Theorem 7.2 of that paper under those assumptions, the random-oracle model, and a knowledge-sound proof system, which for Groth16 means a trusted setup per circuit (Section 9).

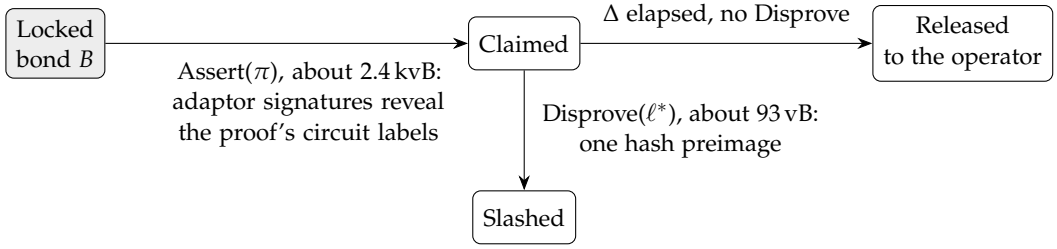
For this backend the setup cost is paid per assertion: the setup garbles a circuit for each Assert, and a garbled circuit is good for one evaluation. It is amortized only across the claims that one assertion covers, so claims have to be batched into one proof per operator and epoch; the per-claim cost is then a share of that setup and of the Assert fee, plus a challenge period measured in blocks. A claim is therefore worth making only if its share of the pool exceeds its proving cost plus its share of the setup, which under Proposition 5 is proportional to the capital behind it, so proof cost sets a minimum capital to participate: roughly the proving cost times the ratio of total registered weight to the pool. This motivates batching over operator epochs before considering frequent small claims. The participation threshold remains backend-dependent: BitVM3 discusses alternatives with different storage costs and challenger assumptions, including estimates awaiting an implemented verifier. A deployment study must identify the backend it measures.

A settlement object's exit can also impose a balance floor. LayerTree's exact exit sizes at depth 16 vary by opcode tier, as Table 11 records [36, Table 5]. The FULL tier additionally has a fee-from-payout exit of 921 model vB. Its other paths require an external fee input under the draft's assumptions. A 1,020 vB exact exit is the FULL value and must not be applied to every tier.

The draft's FULL examples also give about 1,681 vB for a channel-leaf force close and 2,656 vB for a credit-leaf claim followed by an exact exit. At 10 sat/vB these modeled fees are 16,810 and 26,560 sat. A holder's ability to fund the fee input is therefore part of practical exit availability. Similar external liquidity and fee-management questions must be assessed for the other settlement objects under their actual transaction paths; the comparison here has not measured them.

Tier	Model vB	Serialized vB	Model fee at 10 sat/vB	Serialized fee
FULL	1,020	1,030	10,200	10,300
MID	1,355	1,398	13,550	13,980
MIN	1,473	1,518	14,730	15,180
CCV	908	918	9,080	9,180

Table 11: LayerTree exact exits at depth 16, separated by tier and byte convention. Fees are in sat and exclude any additional fee-management transaction or package. These sizes describe a proposed protocol.



Setup: an n-of-n committee presigns this graph and one member deletes its key; the operator garbles the proof verifier, about 41 GB, seven circuits under cut-and-choose. Challenge: any one honest challenger evaluates the circuit off chain in about six minutes and, if the proof is false, learns ℓ^*.

Figure 8: The bond as an object of class E (Proposition 7), with the figures BitVM3 publishes [32]. A true Assert is released after the challenge period; a false one is slashed by any single challenger at a cost that does not depend on the bond. The committee at setup and the per-assertion garbled circuit are the costs Section 6.5 calls bond-scale.

6.6 The bond as an object

A bond that is a number in a record, released or slashed by arithmetic on the same record, admits a negative slash, an over-release, and a bond declared after its owner's wealth is exhausted; the reference model's audit found all three in an earlier implementation. Under the tuple of Table 2 a bond is an object of class E: an on-chain output whose script releases to the operator after Δ unless a valid disprove consumes a connector output first. Its states are those of Figure 8: Locked, Claimed on a valid Assert, then Released after Δ with no Disprove or Slashed on one, with the release condition a statement in $\mathcal{R}_{\text{svc}}$ with k/m above the bonded threshold.

Proposition 7 (enforceable bond)**.** *Under the BitVM3 assumptions, an escrow $E(B, \kappa, \Delta)$ whose release condition is a statement in a SNARK-provable relation is enforceable against an uncooperative operator: a false Assert is slashed by any single honest challenger at a cost independent of B, and a true Assert is released after Δ. The bond is an account in the sense required exactly when the registry records the escrow's outpoint and the journal its state, and release arithmetic is replaced by the object's transitions.*

A native covenant, if one were activated, would replace E by E': no committee, no challenger, $\Delta \approx 0$ for template-enforced spends, and the release condition restricted to what script can check, which excludes SNARK verification without further opcodes. The model keeps E' as a comparison column and depends on nothing in it.

BIP 448 is the concrete candidate for E', and it moves the line less than one might

expect. CSFS lets a script verify a signature on a message, so a release condition of the form "these named attesters signed this window's outcome" can be checked with public attester keys, or an aggregating key, subject to script limits. That is a simpler attestation predicate. It does not by itself enforce the complete admission ledger, unique denominator, deadlines and outcomes required by $\mathcal{R}_{\text{svc}}$, whose compilation remains unimplemented. Trust in the signed statement and its attesters must be specified. $\mathcal{R}_{\text{cap}}$, which hides which object backs a claim, still needs a succinct verifier and so still needs the optimistic game, and of the three mechanisms BitVM3 puts on chain [32] the proposal changes one. The proof is committed by adaptor signatures whose publication reveals the garbled circuit's input labels. That is already a tenth the size of a hash-based commitment, and checking those signatures against a stack message instead of the transaction would not shrink them, so the Assert stays at its published size. The covenant BitVM3 emulates with its committee is a binding between inputs: a withdrawal may spend the reserve only together with a valid Assert. `OP_TEMPLATEHASH` commits to a transaction's outputs and by design not to the other coins it spends [42], so it cannot state that binding, and the committee stays in Γ of the E row. What CSFS does replace is the one-time hash-based signature that holds an operator to a single public input across transactions, since two signatures under one key on different messages are an equivocation that a script can check. That matters where the statement is supplied when the assertion is made and not where it is fixed at setup. We therefore leave the published BitVM3 figures as inputs, unadjusted.

The LayerTree voucher is not an escrow, and the difference says what an escrow is for. A voucher is a signature by a source leaf's owner over that leaf and a new leaf to be carved from it; anyone holding it redeems it alone with a claim, 1,618 vB at $D = 16$, which checks the signature with CSFS and nothing else. It is voidable by the issuer, who can advance the source's counter first, or by the source's evictor after the lease, until the redemption confirms; a cumulative voucher re-signed for the running total needs no contest window, since the latest dominates; the hash-locked form paired with an HTLC on the same hash is an ordered exchange and not an atomic swap, under five rules no script enforces; and a dedicated credit leaf removes the voiding, leaving the issuer's liveness as the holder's exposure. Against Proposition 7: the voucher enforces no statement about the world, slashes nothing, has no deadline because script cannot give it one, and its collateral is the sum it pays. It needs no challenge period, no challenger, no committee and no proof system, because it claims nothing beyond "this leaf may be carved so", whose only risk is the issuer's own voiding or absence. It can carry the connector's payment, and the role of Ark's out-of-round transfer with a different trade, undone by the issuer alone until confirmed against undone by sender and server together; it cannot carry the connector's bond, and the release of $\mathcal{R}_{\text{svc}}$ on an attested service ratio is not a LayerTree predicate.

7 Agent swarms: observers, coordination, and what a coalition can earn

The objects of Sections 4 and 6 were introduced for a population of honest agents. The same objects are available to a population under common control that wants to appear independent, and the question this section asks is what that population can do with them and who would see it. Following Simmons [50], we fix the warden first.

Observer	C channel	V VTXO	E escrow
O_{chain}	opens, closes, splices, force-close scripts	round transactions, exits, offboards; aggregate per round	deposits, Asserts, Disproves, releases
O_{gossip}	`channel_update`, `node_announcement`, policy	nothing per holder	nothing
O_{peer}	HTLC amount, timing, CLTV, preimage on one hop	the HTLC the server forwards on its own channel; the holder is behind the server	nothing
O_{server}	none	every transfer: parties, amount, time, co-signed	operator's own escrow

Table 12: What each observer sees under each object.

7.1 Observers by object

There are four observers, and each object exposes a different trace to each (Table 12). Under the channel no single observer sees a whole path, which is the property the privacy literature studies and which probing and timing attacks partially erode [23, 25, 39]. Under the VTXO one observer sees everything within its Ark: the server co-signs every transfer and so knows sender, receiver, amount, and time for every payment it hosts. A server is a total warden for its holders. The proofs of Section 6 let it not publish what it sees, and a permissionless escrow lets it not be trusted to allocate rewards, but its view is reduced by neither. BIP 448 would change the O_{peer} row at the margin: its authors expect rebindable signatures to simplify the move to point time-locked contracts, under which two hops of one payment no longer share a hash to correlate on, while amounts and timing remain. Under T the operator co-signs grafts and updates and sees every transition of the output, as does anyone, but a voucher in flight and a payment in a channel leaf are not shown to it: O_{server} is partial where under V it is total, and O_{chain} sees one output's whole spend chain.

7.2 Forms of coordination

Explicit messages can use application-defined onion TLV records or protocol-visible metadata, subject to the relevant payload, relay and bandwidth limits [27, 28]. A capacity estimate must specify the observer and permitted transport. Implicit coordination uses shared observations or conventions; public events can supply information even when the agents send no private message. Capital coordination changes who claims a reward, places funds or supplies demand. The reward construction below constrains one part of the last activity. Each mode needs a separate experimental outcome.

7.3 What a coalition can earn

Let a coalition share a controller and coordinate its claims and capital placement. The authenticated reward rule gives a bound on one component of its receipts.

Proposition 8 (capital-proportional allocation, conditional)**.** *Suppose Propositions 5 and 6 hold for a complete eligible set of disjoint exposures. A fixed pool P allocated proportionally to accepted weight pays a coalition of weight W_c exactly PW_c/W before integer rounding, where*

$W > 0$ is total accepted weight. Splitting the same admitted exposure among more claimant identities cannot increase W_c.

This conclusion follows from the allocation rule. It bounds this reward pool under its eligibility assumptions. A coalition can also earn routing fees, change its service, trade, or affect demand; those channels require additional models. Table 1 supplies an empirical comparator for one placement heuristic, rather than an upper bound on those gains. Its stationary effect is 6.13 success points and its shifted effect is estimated with uncertainty around 0.44 points. Neither quantity is a measure of coalition profit.

Service rewards add a separate exposure. An outcome ledger can authenticate requests without establishing that their counterparties are independently controlled. On the existing coalition fixtures, a network-wide gross-to-net churn ratio fails to separate the labeled patterns: benign multilateral commerce nets strongly. Netting restricted to direct attesters catches a reciprocal pair and misses a four-member relay. A repeated-cycle statistic separates deliberately conspicuous fixtures, where the injected signal is hundreds of times larger than background. Its performance on ordinary recurring commerce and adaptive coalitions remains unmeasured. Accordingly this paper makes no claim that the proposed service reward is collusion-proof.

Observation and authenticated accounting determine which of these questions a monitor can test. A capacity-versus-detectability experiment for payment-shaped messages would address another channel of coordination; its result would complement the allocation statement, whose scope is the fixed reward pool.

7.4 The warden's statistics

Formally the warden observes a trace from one row of Table 12 and compares H_0, independently controlled agents under a declared demand process, with H_1, common control of a subset. No alternative trace distribution is specified here, so a likelihood-ratio test has not been defined. Candidate summaries include repeated-cycle share, repeated-nullifier submissions, and inconsistency between an attested service ratio and independently observed demand. Repeated nullifiers identify repeated credential/object claims; ordinary retries can produce them, and duplicated economic capital can evade them through successor identifiers. They are not by themselves a calibrated detector of control. A server can compute cycle statistics completely only for transfers within its observation domain. The channel fixtures instead expose 11–13% of events to a partial observer. Cross-server fixtures must test what remains measurable when no observer sees the full cycle. Under LayerTree, a further candidate is activity on the output's public spend chain, subject to what the chosen opcode tier reveals. Such a statistic supplies no exit guarantee; the queueing conditions of Section 4.7 remain necessary.

7.5 What a swarm does with the objects

The analysis so far assumes the coalition uses the objects as a holder. Six further possibilities change the picture, and each is stated as a conjecture with the experiment that would test it.

The swarm runs the server. A commonly controlled population can operate an Ark server if it supplies the capital, channels and continuous service the implementation requires. Transfers within that Ark need not be published individually on chain; the server

observes them. This can reduce the public trace relative to separately announced Lightning nodes, at the cost of operating and funding the server. An external monitor has no automatic access to that internal record. Whether round cadence or output distributions identify common control is an empirical fingerprinting question; a chain observer cannot directly measure unreported out-of-round churn.

Ramps remain. Lightning receipts cross the server's gateway channels, and boardings and exits have on-chain transactions. Public channels expose advertised capacity, while peers observe their own payment legs. Those observations do not reveal every flow, identify its ultimate beneficiary, or uniquely establish common control. A monitor would have to specify which perimeter metadata it obtains and evaluate inference from that trace. Private channels and multi-hop routing further limit a public observer's view.

Cross-server circulation. Two swarm-run Arks paying each other over Lightning look like two independent servers with ordinary bilateral settlement; each server's view is total inside and nil across, and a cycle through both is two intra-Ark legs no outsider sees plus two Lightning legs that look like server-to-server flow. The repeated-cycle statistic needs an observer who sees the cycle, and with two servers no single row of Table 12 does. This is the fixture to build.

The swarm is the channel. Under BIP 448 a swarm needs no server to obtain what the swarm-run server gave it. A multiparty channel whose members are all the swarm's agents is funded by one output that a key-path spend makes indistinguishable from a single signer's, has no rounds and so no round cadence to fingerprint, appears in gossip as one channel or as none, and settles every transfer inside the swarm as a reassignment of balances that only members see. The perimeter argument survives, since value still enters and leaves over some member's channels to the rest of the network, and the fixed-pool allocation statement of Proposition 8 still applies under its eligibility assumptions. What does not survive is the chain-fingerprinting route to the question of which servers are swarm-run, because there is no server. The object cuts the other way when membership is mixed: every member signs every state, so each honest member of a hyperedge is a total observer of it, and a monitor that wants the capital-coordination signal should want to be a member. Table 12 would carry an O_{member} row under C_k that reads like the O_{server} row under V, held by $k+1$ parties instead of one.

The registry's privacy is symmetric. $\mathcal{R}_{\text{cap}}$ hides its witness under the proof system's zero-knowledge assumptions, while exposing the claim value, root and nullifier. A commonly controlled population can receive its proportional reward without a registry rule that identifies all its members as one principal. Propositions 5 and 6 constrain accepted exposure, not the number of controllers. Submission metadata, values and linking across epochs may nevertheless reveal structure. The construction therefore makes no general anonymity or control-indistinguishability claim.

Enforceable side-contracts. Observable deviations and credible punishments matter to the stability of collusive arrangements [12, 51]. A funded escrow can enforce a specified verifiable predicate if its setup and challenge assumptions hold. Common control of the setup committee supplies no evidence that keys were deleted and gives an outside

counterparty no independent assurance. Agents controlled by one principal can already follow an internal policy, so the economically informative experiment uses parties with distinct incentives and compares their behavior with and without an externally enforceable commitment. The complete service predicate and the relevant setup remain to be implemented.

Shared weights and shared control. Agents using the same model can choose correlated actions because their policies or observations overlap. Different prompts, budgets and random seeds can also make their actions diverge. A trace-based detector must therefore test shared-model, independently controlled agents as a negative class. Observational indistinguishability follows when the two hypotheses induce the same trace distribution; sharing weights alone does not establish that equality. The proposed experiment fixes the observation channel and compares both hypotheses with held-out tasks and budgets.

The lever is the authority field. Every case above assumes the swarm holds its own keys. The tuple has the field where the safety lever sits: $\mathcal{A}$, the spend authority. One possible policy design includes a principal-held co-signer and bounded delegation, so that an agent spends within a budget and window without a co-signature and beyond it needs one. A principal can apply policy in its agent wallet or signing service. An Ark server supplies an additional co-signing point for transfers it serves; its effectiveness depends on whether the principal controls that server and whether the agent can move funds elsewhere. The observer table should carry an $O_{\text{principal}}$ row, since for alignment purposes the observer who matters most is the operator the agents nominally work for, and the failure of interest is a population that routes around its own principal's view. BIP 448 gives the lever a second seat and a weaker one. Its proposers describe CSFS as enabling delegation: a principal signs a message naming an agent's key and a script checks that signature before the agent's, so that delegated spend authority becomes expressible on any Taproot output and not only where a server co-signs. It falls short of the delegation asked for above in two respects. Script can delay a spend but cannot expire one, so a window closes only when the principal moves the coin; and the proposal introspects no amounts except through a template that fixes every output, so a budget is a set of templates the principal has signed. In a hyperedge the principal has a simpler option, which is to be a member. Key aggregation gives the lever a third seat, and one available now: with MuSig2 the agent's key in any of these objects can be a two-of-two of agent and principal, and under the nested draft a group's key can stand wherever one member's does, so the principal signs every spend with no script and without the co-members knowing (Section 5.6). The same fact serves a swarm, whose members' keys can share a signer that no observer can see. A LayerTree operator is a weaker enforcement point than an Ark server, and so a weaker evasion point for a swarm that runs one: under the draft's contention assumption [C] and the required service and fee conditions, it cannot delay an eligible exit beyond that bound, move or reduce another leaf, or freeze value it did not graft and fund; what it can refuse is a graft and its signature on an update.

8 Experiments to preregister

The archived campaigns, the October 21 finite witnesses and the new sensitivity campaign are completed work. The following items are prospective. Each new run must freeze implementation commits, initial-state hashes, allowed observations and stopping rules before

its holdout data are inspected. Pilot calibration and confirmation seeds must be separated. The detailed protocol is `audit/peer_review/testnet4_plan.md`.

1. **Trace one Ark recovery cycle on regtest.** Use an owned Bark/captaind stack and a pinned Lightning implementation. Record each asset and liability through a successful send, a failed send, a receive, a refresh and an expiry sweep. Separate reserve reassignment from recovery of the original backing. Log transaction identifiers, VTXO ancestry, lock conditions and spendability at each height. Expiry recovery is the documented baseline; any earlier reuse must identify a valid transaction path and the asset that funds it. The accounting check fails if an obligation disappears without a corresponding settlement or if any capital is counted twice.

2. **Calibrate at equal service.** Model 432-block receive-pool outputs separately from 4032-block refreshed outputs, include their residual ages, and charge idle reserves as well as occupied locks. Choose budgets and forecast parameters on pilot seeds to meet a stated delivery target, then hold them fixed on independent confirmation seeds. Report delivery, unmet satoshis, occupied and reserved capital-time, terminal obligations, and duration per offered and per delivered volume. Repeat with correlated demand and shifts. Report a ranking only at the declared service target; a single favorable duration ratio is insufficient.

3. **Measure the operational clocks on testnet4.** After local reconciliation passes, run a bounded study on four controlled Lightning nodes in a diamond, with a proposed ceiling of 200 payments and 48 hours. Use a funding budget fixed before launch. Capture receipt-time timestamps at the observer, funding inclusion and activation depth, attempted routes, HTLC reservations, settled balances and payment outcomes. Miner header timestamps do not measure observed latency. Identical-state policy counterfactuals stay on regtest, where state can be restored. This experiment measures implementation behavior on a public test chain; it does not estimate mainnet fee economics or the behavior of an agent population.

4. **Measure proposed-object costs in an enforcing environment.** BIP 448 and LayerTree require a local node or custom network implementing their proposed rules. Public testnet4 acceptance alone cannot validate an inactive opcode's semantics. Reproduce serialized exits by tier and measure renewal, graft and fee-management packages. For contention, use controlled regtest miners and record the actual service rate, conflicting transitions and reconstruction delays. For hyperedges, vary group size, within-group payment share and correlated departures. Estimate the cost of churn alongside capital savings.

5. **Implement the reward and observation experiments.** Compile the restricted $\mathcal{R}_{\text{cap}}$ relation with canonical credentials and disjoint backing, benchmark it on named hardware, and test identifier churn and alternative-authority witnesses. Specify a complete admitted-request ledger before using $\mathcal{R}_{\text{svc}}$ as a reward denominator. Evaluate repeated-cycle statistics on benign recurring commerce and mixed-server fixtures, and include independently controlled agents sharing one model. Report false positives at a threshold fixed on separate calibration data. A real setup and challenge path is required before the escrow state machine can support an end-to-end enforceability claim.

9 Limitations

The cut identities are accounting statements. They do not certify routability, choose a routing policy or bound the profit from coordination. The archived policy contrasts are conditional on sampled subgraphs of one 2023 snapshot, synthetic hidden state and the tested route catalog. Hash-order sensitivity limits exact regeneration of historical runs, although the retained rows and their statistics are verifiable. The new demand-seed campaign is exploratory work in the reference model.

The settlement grid uses a single lifetime and an unconstrained fronting reserve, charges occupied locks, and sizes the server's holder-facing stock after its flow is known. In trailing cells, failed server forecasts do not feed back into holder execution. It lacks fee markets, reserve financing, HTLC concurrency, reorganization effects and actual Bark transaction construction. Current documentation identifies expiry recovery and different receive-pool lifetimes; measuring those paths and matching service levels precede a deployment comparison.

The reward arguments assume an authenticated eligible set with disjoint backing and one canonical nullifier credential per admitted object. The reference verifier checks a simulated witness, with no implemented zero-knowledge circuit. A complete service denominator, independent counterparty control and robust coalition detection remain open. The allocation proposition bounds one fixed reward pool and supplies no general earnings bound.

An optimistic escrow additionally requires the cited BitVM3 setup and challenger assumptions. Groth16 requires valid circuit-specific setup parameters; retained setup randomness can enable forged proofs [21, 24]. An honest key-deletion participant in the transaction-signing setup and a valid proof-system setup are separate requirements. The published Yao/Groth16 cost figures are backend inputs, not measurements of the relations proposed here.

BIP 448 and the LayerTree opcode tiers remain proposals. The hyperedge analysis assumes a state protocol that provides the stated membership and exit behavior; its independent departure model does not describe coordinated failures. LayerTree byte estimates include both model and serialized conventions where the source provides them, while deployment package costs remain unmeasured. Queue delay requires the minimum-service and bounded-overtaking assumptions of Section 4.7. The historical testnet4 headers validate only their recorded header properties and parent links. No live payment or recovery experiment accompanies this revision.

10 Conclusion

Settlement objects determine where payment capacity is committed and how long it remains unavailable. The prefix cut identity records that constraint for bilateral and multiparty channels. Routing still matters through the balances a successful path leaves behind. In the archived workloads, the tested demand-aware placement policy has a larger stationary success effect than the tested online routing heuristic; the comparison remains specific to those policies and inputs.

The corrected reference model finds useful pooling at small group sizes under independent zero-drift demand. Its exploratory 24-seed campaign gives a mean 1.467 ratio between bilateral-channel and twenty-agent-hyperedge capital-time at matched oracle service. Ark's comparison depends materially on fronting duration, and the documented

expiry path must be calibrated with its receive-pool and refresh lifetimes. LayerTree introduces recurring bytes and queue service into the same accounting question.

An eligible-set registry can make a fixed reward pool proportional to disjoint authenticated exposure under a canonical credential rule. Extending that statement to service quality or coalition earnings requires further evidence. The next implementation result should reconcile an entire capital-recovery cycle and then compare budgets at the same service level, with testnet4 supplying observed operational clocks after that local accounting is settled.

References

[1] Kenneth J. Arrow, Theodore Harris, and Jacob Marschak. Optimal inventory policy. *Econometrica*, 19(3), 1951.

[2] Moshe Babaioff, Shahar Dobzinski, Sigal Oren, and Aviv Zohar. On Bitcoin and red balloons. In *ACM Conference on Electronic Commerce (EC)*, 2012.

[3] bcSatoru. The price of being receivable: What inbound liquidity actually costs on Lightning, 2026. Spark research note, September 21, 2026, on mid-2026 market rates: liquidity ads 0.5 to 1.5 percent a year, Lightning Pool 1.5 to 3, Magma about 2.6 for channels of a bitcoin and more and over 4 under a million sat. Advertised rates, not executed trades. https://www.spark.money/research/lightning-inbound-liquidity-acquisition-cost, accessed October 16, 2026.

[4] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, 2014.

[5] Ferenc Béres, István András Seres, and András A. Benczúr. A cryptoeconomic traffic analysis of Bitcoin's Lightning Network. *Cryptoeconomic Systems*, 1(1), 2021.

[6] Nik Bhatia. The Lightning Network reference rate, 2019. Medium, March 11, 2019, part 4 of the series of that name. Defines the node accrual rate $[(p+f)/p]^{52{,}560/n} - 1$, routing fees f on average channel balance p over n blocks, annualised, and the reference rate as an aggregate of disclosed node rates; its examples are 0.25 and 0.18 percent. https://timevalueofbtc.medium.com/the-lightning-network-reference-rate-98e41a9dadfa, accessed October 16, 2026.

[7] Bitcoin Optech. Ark, . Topic page, https://bitcoinops.org/en/topics/ark/, accessed September 30, 2026.

[8] Bitcoin Optech. OP_CHECKTEMPLATEVERIFY, . Topic page, https://bitcoinops.org/en/topics/op_checktemplateverify/, accessed September 30, 2026.

[9] Bitcoin Optech. Eltoo, . Topic page, https://bitcoinops.org/en/topics/eltoo/, accessed October 5, 2026.

[10] Conrad Burchert, Christian Decker, and Roger Wattenhofer. Scalable funding of Bitcoin micropayment channel networks. *Royal Society Open Science*, 5(8), 2018.

[11] Christian Cachin. An information-theoretic model for steganography. In *Information Hiding (IH)*, 1998.

[12] Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110 (10), 2020.

[13] Lin William Cong, Xi Li, Ke Tang, and Yang Yang. Crypto wash trading. *Management Science*, 69(11), 2023.

[14] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. eltoo: A simple layer2 protocol for Bitcoin, 2018. Whitepaper.

[15] John R. Douceur. The Sybil attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.

[16] Patrick B. Dugan. Spiral: archived campaigns, audit, and reference model for agent liquidity on Lightning, 2026. Repository, https://github.com/patrickdugan/Spiral. Public-topology campaign at commit a0a3140; audit at commit 618a12e; reviewed paper baseline at commit 6a9df36; October 21, 2026 corrections and reproduction instructions in audit/peer_review and paper/peer_review_changes.md.

[17] Patrick B. Dugan, Daniel P. Pizarro, and Lihki J. Rubio. Graph re-write using ghost nodes for target value rebalancing, 2021. U.S. Patent Application Publication US 2021/0004796 A1, published January 7, 2021; application 16/920,416 filed July 2, 2020.

[18] Gary D. Eppen. Effects of centralization on expected costs in a multi-location newsboy problem. *Management Science*, 25(5), 1979.

[19] Beulah Evanjalin. BIP draft: NestedMuSig2 for BIP340-compatible signatures, 2026. bitcoin/bips pull request 2297, opened September 21, 2026; head 4523ee0 of October 15, 2026. Our review of the September head is linked from the pull request. https://github.com/bitcoin/bips/pull/2297.

[20] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 1956.

[21] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*, 2016.

[22] Paolo Guasoni, Gur Huberman, and Clara Shikhelman. Lightning network economics: Channels. *Management Science*, 2024.

[23] Jordi Herrera-Joancomartí, Guillermo Navarro-Arribas, Alejandro Ranchal-Pedrosa, Cristina Pérez-Solà, and Joaquin Garcia-Alfaro. On the difficulty of hiding the balance of Lightning Network channels. In *ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2019.

[24] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash protocol specification, 2016. Electric Coin Company, versions through 2023.

[25] George Kappos, Haaroon Yousaf, Ania Piotrowska, Sanket Kanjalkar, Sergi Delgado-Segura, Andrew Miller, and Sarah Meiklejohn. An empirical analysis of privacy in the Lightning Network. In *Financial Cryptography and Data Security (FC)*, 2021.

[26] Burak Keceli. Ark: An alternative privacy-preserving second layer solution, 2023. bitcoin-dev mailing list, May 2023.

[27] Lightning protocol contributors. BOLT 4: Onion routing protocol, . https://github.com/lightning/bolts/blob/master/04-onion-routing.md, accessed September 30, 2026.

[28] Lightning protocol contributors. BOLT 7: P2P node and channel discovery, . https://github.com/lightning/bolts/blob/master/07-routing-gossip.md, accessed September 30, 2026.

[29] Lightning Service Provider Specification group. LSPS2: JIT channel negotiation (bLIP-52), 2024. https://github.com/lightning/blips.

[30] Robin Linus. BitVM: Compute anything on Bitcoin, 2023. Whitepaper, https://bitvm.org.

[31] Robin Linus, Lukas Aumayr, Alexei Zamyatin, Andrea Pelosi, Zeta Avarikioti, and Matteo Maffei. BitVM2: Bridging Bitcoin to second layers, 2024. Whitepaper, https://bitvm.org.

[32] Robin Linus, Ioannis Alexopoulos, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and David Tse. BitVM3: Efficient Bitcoin bridges via garbled circuits, 2026. IACR ePrint 2026/933, revised June 8, 2026.

[33] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In *CRYPTO*, 2021.

[34] Jonas Nick, Tim Ruffing, and Elliott Jin. BIP 327: MuSig2 for BIP340-compatible multi-signatures, 2022. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki.

[35] Utz Nisslmueller, Klaus-Tycho Foerster, Stefan Schmid, and Christian Decker. Toward active and passive confidentiality attacks on cryptocurrency off-chain networks. *arXiv preprint arXiv:2003.00003*, 2020.

[36] Olaoluwa Osuntokun. LayerTree: A shared Bitcoin UTXO with an O(1) unilateral exit, 2026. Draft dated October 15, 2026, commit ba8422e. https://layertree.tech/paper/main.pdf, accessed October 16, 2026.

[37] Rene Pickhardt and Mariusz Nowostawski. Imbalance measure and proactive channel rebalancing algorithm for the Lightning Network. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2020.

[38] Rene Pickhardt and Stefan Richter. Optimally reliable & cheap payment flows on the Lightning Network. *arXiv preprint arXiv:2107.05322*, 2021.

[39] Matteo Romiti, Friedhelm Victor, Pedro Moreno-Sanchez, Peter Sebastian Nordholt, Bernhard Haslhofer, and Matteo Maffei. Cross-layer deanonymization methods in the Lightning protocol. In *Financial Cryptography and Data Security (FC)*, 2021.

[40] Steven Roose and Erik De Smedt. Evolving the Ark protocol using CTV and CSFS, 2025. Delving Bitcoin, April 15, 2025. https://delvingbitcoin.org/t/evolving-the-ark-protocol-using-ctv-and-csfs/1602.

[41] Jeremy Rubin. BIP-119: CHECKTEMPLATEVERIFY, 2020. https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki.

[42] Gregory Sanders, Antoine Poinsot, and Steven Roose. BIP 446: OP_TEMPLATEHASH, 2026. Draft, assigned February 6, 2026. https://github.com/bitcoin/bips/blob/master/bip-0446.md, accessed October 5, 2026.

[43] Gregory Sanders, Antoine Poinsot, and Steven Roose. BIP 448: Taproot-native (re)bindable transactions, 2026. Draft, assigned March 11, 2026. https://github.com/bitcoin/bips/blob/master/bip-0448.md, accessed October 5, 2026.

[44] Thomas C. Schelling. *The Strategy of Conflict*. Harvard University Press, 1960.

[45] Second. Lightning liquidity: Ark servers and LSPs compared, 2025. https://second.tech/blog/ark-liquidity-research-01, May 1, 2025.

[46] Second. Forfeits & connectors, 2026. Documentation, https://second.tech/docs/learn/forfeits, accessed September 30, 2026.

[47] Second. Lightning payments, 2026. Implementation documentation, https://second.tech/docs/learn/payments-lightning, accessed October 21, 2026. Specifies expiry recovery on successful sends and separate 432-block receive-pool lifetimes.

[48] Second. Ark liquidity, 2026. Implementation documentation, https://second.tech/docs/learn/liquidity, accessed October 21, 2026.

[49] Second. Ark VTXOs, 2026. Documentation, https://second.tech/docs/learn/vtxo, accessed September 30, 2026.

[50] Gustavus J. Simmons. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of CRYPTO '83*, 1984.

[51] George J. Stigler. A theory of oligopoly. *Journal of Political Economy*, 72(1), 1964.

[52] Sergei Tikhomirov, Rene Pickhardt, Alex Biryukov, and Mariusz Nowostawski. Probing channel balances in the Lightning Network. *arXiv preprint arXiv:2004.00333*, 2020.

[53] Friedhelm Victor and Andrea Marie Weintraud. Detecting and quantifying wash trading on decentralized cryptocurrency exchanges. In *The Web Conference (WWW)*, 2021.

A Notation

Symbol	Meaning
$\rho = (\mathcal{A}, v, F, X, \Lambda, \Gamma, \tau)$	settlement object: authority, value, finality clock, exit, liveness, counterparty set, expiry
$\ell^U, \ell^C, \ell^V, \ell^E$	holdings by class
σ	settlement event (from-class, to-class, amount, clock condition)
O_A, I_A	cumulative outward, inward crossings of ∂A
S_A, S'_A, K_A, K'_A	outbound and inbound stock on ∂A; stock added by capacity events
$\hat{f}_a, \hat{f}_S$	forecast inbound toward agent a; forecast peak aggregate holdings at server S
P_S, W_S, B_S^F	holder-facing stock, unrecovered advances, fronting reserve
F_S, C_S	cumulative advances and recoveries in the fronting ledger
$\mathcal{D}_{\text{obj}}(D, H)$	liquidity duration: locked capital-time per unit delivered
$\mathcal{R}_{\text{cap}}, \mathcal{R}_{\text{svc}}$	unique-exposure and service relations
$\text{nf}, \text{cm}, \text{root}_n, k_\rho$	nullifier, claimant commitment, settled-set root, canonical credential
$E(B, \kappa, \Delta)$	escrow with bond B for connector κ and challenge period Δ
$\mathcal{M}_i, A_i, h$	member set of hyperedge i, its agents, and its hub
$\hat{f}_{A_i}$	forecast peak of the total the agents of hyperedge i hold
R_a	capacity moved toward agent a by reallocation inside a factory, off chain
$\mathcal{D}_k$	liquidity duration of hyperedges of k agents and a hub; $\mathcal{D}_1 = \mathcal{D}_C$
T, P_a, G	the LayerTree leaf; the operator's pre-funded credit leaf for member a; graft value it fronts
T_u, T_ℓ, W	top-up period, lease period and settle window under T
κ, β, $\Delta(q)$	assumed minimum services per interval; overtaking budget; conditional delay
b_T	on-chain bytes per member per block under T
$O_{\text{chain}}, O_{\text{gossip}}, O_{\text{peer}}, O_{\text{server}}$	observer rows

B Reference model

The TypeScript reference model lives under `model/` and uses Node's built-in modules. It covers the settlement ledger, fronting and recovery, channel and hyperedge sizing, the single-credential epoched registry, escrow transitions and fixture statistics. The proof verifier is simulated. LayerTree's accounting tables are reproduced from the cited draft; its covenant scripts are not executed by this model.

The October 21 review in `audit/peer_review/` includes finite counterexamples, the archived-data audit and the 24-seed sensitivity. Forecasts now follow the replay's receipt-first and all-or-nothing spend rules. Regression checks cover 1,024 five-round demand paths and exact recovery integrals; they do not prove distributed correctness.

Run `model/grid.ts` with Node's type stripping to regenerate `grid.tex`. Sensitivity, build and verification commands are in `audit/peer_review/README.md`. Earlier manuscripts and archived results remain preserved.